

Министерство образования Иркутской области
Государственное бюджетное профессиональное образовательное учреждение
Иркутской области
«Иркутский авиационный техникум»
(ГБПОУИО «ИАТ»)

ДП.09.02.07-5.26.222.10 ПЗ

УТВЕРЖДАЮ

Зам. директора по УР, к.т.н.

_____ Е.А. Коробкова

ВЕБ-ПРИЛОЖЕНИЕ
СПОРТИВНОГО КЛУБА «АВИАТОР»

Нормконтроллер:	_____	(Н.Р. Огородникова)
	(подпись, дата)	
Руководитель:	_____	(П.Н. Чернигов)
	(подпись, дата)	
Студент:	_____	(Г.М. Зимин)
	(подпись, дата)	

Иркутск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Предпроектное исследование	5
1.1 Описание предметной области	5
1.2 Анализ инструментов, используемых в разработке программного продукта..	9
2 Техническое задание на разработку программного продукта.....	17
3 Проектирование программного продукта	18
3.1 Архитектура программного продукта.....	18
3.2 Функциональное и структурное проектирование.....	19
3.3 Проектирование базы данных.....	24
3.4 Проектирование пользовательского интерфейса.....	30
4 Реализация программного продукта	41
4.1 Реализация базы данных	41
4.2 Реализация серверной части	43
4.3 Реализация клиентской части	47
5 Тестирование программного продукта	50
5.1 Функциональное тестирование.....	50
5.2 Нагрузочное тестирование	54
6 Руководство пользователя программного продукта	57
6.1 Инструкция по установке и запуску программного продукта	57
6.1 Руководство пользователя для роли Студент	58
6.2 Руководство пользователя для роли Преподаватель.....	64
ЗАКЛЮЧЕНИЕ	73
Список используемых источников.....	75
Приложение А – Техническое задание	77
Приложение Б	89

					ДП.09.02.07-5.26.222.10.ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Зимин Г.М.			ВЕБ-ПРИЛОЖЕНИЕ СПОРТИВНЫЙ КЛУБ «АВИАТОР» пояснительная записка	Лит.	Лист	Листов
Провер.		Чернигов П.Н.					2	98
Реценз.						ГБПОУИО «ИАТ» ИС-22-2		
Н. Контр.		Огородникова Н.Р.						
Утверд.		Коробкова Е.А.						

ВВЕДЕНИЕ

В условиях цифровой трансформации образования и повышения внимания к формированию здорового образа жизни студентов возникает необходимость в эффективных цифровых инструментах для организации спортивной деятельности. Одним из таких инструментов является веб-приложение спортивного клуба «Авиатор», предназначенное для информирования студентов Государственного бюджетного профессионального образовательного учреждения Иркутской области «Иркутский авиационный техникум» (далее – ГБПОУИО «ИАТ», техникум) о спортивной жизни образовательной организации и стимулирования интереса к физической культуре и спорту.

Веб-приложение должно обеспечивать предоставление актуальной информации о спортивной жизни техникума: новостях, спортивных достижениях, доступных секциях, тренировках, соревнованиях и командах. Кроме того, система должна поддерживать запись студентов на тренировки, подачу заявок на участие в соревнованиях, подачу заявок на вступление в спортивные команды, а также просмотр истории участия и результатов спортивной деятельности.

Разработка веб-приложения направлена на повышение доступности информации о спортивной деятельности, увеличение вовлечённости студентов и формирование мотивации к активному образу жизни. Проект выступает как средство автоматизации управления спортивными мероприятиями и как элемент цифровой инфраструктуры техникума.

Ключевые характеристики разрабатываемого программного продукта:

- поддержка различных спортивных дисциплин;
- предоставление актуальной информации о тренировках, соревнованиях, командах и новостях спортивного клуба;
- возможность подачи заявок на участие в спортивных мероприятиях и вступление в команды;

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 3
Изм.	Лист	№ докум.	Подпись	Дата		

- ведение сведений об участниках, командах, тренировках и соревнованиях;
- просмотр истории участия и спортивных результатов;
- автоматизация организационных процессов спортивного клуба.

Цель дипломного проекта – разработать веб-приложение спортивного клуба «Авиатор» для ГБПОУИО «ИАТ», предназначенное для предоставления информации о спортивной деятельности техникума, автоматизации записи на тренировки, подачи заявок на соревнования и команды, а также учёта спортивных достижений.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести предпроектное исследование предметной области, определить основные процессы, объекты, участников и их роли.
2. Выполнить анализ инструментальных средств, используемых при разработке программного продукта.
3. Разработать техническое задание на создание веб-приложения.
4. Спроектировать программный продукт, включая архитектуру, функциональную структуру, базу данных и пользовательский интерфейс.
5. Реализовать серверную часть, клиентскую часть и базу данных веб-приложения.
6. Провести функциональное и нагрузочное тестирование программного продукта.
7. Разработать проектную документацию, включая инструкцию по установке и руководство пользователя для предусмотренных ролей.

Практическая значимость дипломного проекта заключается в возможности применения разработанного веб-приложения для централизованного информирования студентов о спортивной жизни техникума, автоматизации работы с тренировками, соревнованиями и командами, а также упрощения учёта участия студентов в спортивной деятельности.

1 Предпроектное исследование

1.1 Описание предметной области

Предметной областью дипломного проекта является организация спортивной деятельности в ГБПОУИО «ИАТ» с использованием веб-приложения спортивного клуба «Авиатор». В рамках данной предметной области рассматриваются процессы информирования студентов о спортивной жизни техникума, ведения сведений о спортивных секциях, командах, тренировках, соревнованиях, заявках на участие и спортивных результатах.

Спортивная деятельность техникума включает проведение тренировочных занятий, организацию соревнований, формирование спортивных команд, публикацию новостей, учёт участников и фиксацию результатов спортивных мероприятий. Указанные процессы требуют своевременного обновления информации, распределения прав между пользователями и хранения данных в едином веб-приложении.

При отсутствии специализированного программного продукта информация о спортивной деятельности может храниться в разрозненных источниках: объявлениях, документах, таблицах, сообщениях и устных уведомлениях. Такой подход затрудняет поиск актуальной информации о тренировках, соревнованиях, командах и результатах, а также усложняет формирование заявок студентов на участие в спортивных мероприятиях.

Основными процессами предметной области являются:

- публикация и просмотр новостей спортивного клуба;
- ведение сведений о видах спорта;
- формирование и сопровождение спортивных команд;
- создание и проведение тренировочных сессий;
- запись студентов на тренировки;
- создание и проведение соревнований;

- подача и обработка заявок на участие в соревнованиях;
- подача и обработка заявок на вступление в команды;
- фиксация результатов соревнований;
- просмотр истории участия студентов в тренировках, командах и соревнованиях;
- формирование отчетов о спортивной деятельности спортивного клуба.

В предметной области выделяются следующие роли пользователей:

- неавторизованный пользователь;
- студент;
- преподаватель.

Неавторизованный пользователь имеет доступ к публичной части веб-приложения, просматривать общую информацию о спортивном клубе, новости, виды спорта, команды и результаты соревнований, доступные в общем доступе в веб-приложении.

Студент является основным пользователем веб-приложения, просматривать новости, соревнования, тренировки, команды и результаты, записываться на тренировки, подавать заявки на участие в соревнованиях, подавать заявки на вступление в команды, а также просматривать собственный профиль и историю участия в спортивной деятельности.

Преподаватель является пользователем с расширенными правами, осуществляет управление спортивной информацией, создаёт и редактирует новости, виды спорта, команды, тренировки и соревнования, рассматривает заявки студентов, формирует составы команд, управляет результатами соревнований и просматривает сведения о студентах.

Основными объектами предметной области являются:

- пользователь;
- новость;
- вид спорта;

- команда;
- заявка на вступление в команду;
- тренировочная сессия;
- запись на тренировку;
- соревнование;
- заявка на участие в соревновании;
- категория соревнования;
- локация;
- результат соревнования.

Объект «Пользователь» содержит сведения об участнике системы. К основным атрибутам пользователя с ролью студент относятся идентификатор, логин, имя, фамилия, отчество, роль, учебная группа, статус активности учётной записи и статус физического организатора.

Объект «Новость» содержит сведения об информационной публикации спортивного клуба. К основным атрибутам новости относятся идентификатор, заголовок, описание, дата публикации, статус и сведения об авторе публикации.

Объект «Вид спорта» содержит сведения о спортивной дисциплине, доступной в спортивном клубе. К основным атрибутам вида спорта относятся идентификатор, наименование и описание.

Объект «Команда» содержит сведения о спортивной команде. К основным атрибутам команды относятся идентификатор, наименование, описание, связанный вид спорта и состав участников.

Объект «Заявка на вступление в команду» содержит сведения о запросе студента на включение в состав спортивной команды. К основным атрибутам заявки относятся идентификатор, студент, команда, статус заявки, дата создания и причина отклонения при наличии.

Объект «Тренировочная сессия» содержит сведения о тренировке. К основным атрибутам тренировочной сессии относятся идентификатор, вид спорта, команда,

название, описание, дата и время начала, дата и время окончания, статус и место проведения.

Объект «Запись на тренировку» содержит сведения о регистрации студента на тренировочную сессию. К основным атрибутам записи относятся идентификатор, тренировочная сессия, студент и дата регистрации.

Объект «Соревнование» содержит сведения о спортивном соревновании. К основным атрибутам соревнования относятся идентификатор, вид спорта, команда, название, описание, дата начала, дата окончания, статус, категория и локация.

Объект «Заявка на участие в соревновании» содержит сведения о запросе пользователя на участие в соревновании. К основным атрибутам заявки относятся идентификатор, пользователь, соревнование, статус заявки, пользователь, принявший решение, причина отклонения и дата принятия решения.

Объект «Категория соревнования» содержит сведения о классификации соревнований. К основным атрибутам категории относятся идентификатор, наименование и описание.

Объект «Локация» содержит сведения о месте проведения тренировки или соревнования. К основным атрибутам локации относятся идентификатор, наименование, описание и организатор при необходимости.

Объект «Результат соревнования» содержит сведения об итогах завершённого соревнования. К основным атрибутам результата относятся идентификатор, соревнование, команда, занятое место и признак нахождения в архиве.

Таким образом, предметная область включает совокупность процессов, связанных с информированием пользователей о спортивной жизни техникума, организацией тренировок и соревнований, формированием команд, обработкой заявок и учётом спортивных результатов. Разработка веб-приложения позволит централизовать данные спортивного клуба, упростить взаимодействие студентов и преподавателей и повысить доступность информации о спортивной деятельности.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 8
Изм.	Лист	№ докум.	Подпись	Дата		

1.2 Анализ инструментов, используемых в разработке программного продукта

На данном этапе определён и обоснован перечень инструментальных средств, применяемых при проектировании, реализации, тестировании и сопровождении веб-приложения спортивного клуба «Авиатор». Выбор ориентирован на используемые информационные технологии реализованных информационных ресурсов техникума (корпоративная сеть, LDAP, сервер MySQL), удобство разработки одним разработчиком и возможность дальнейшей поддержки.

Веб-приложение необходимо для ведения учёта студентов, команд, соревнований, тренировок и новостей клуба. Это возможно реализовать с помощью базы данных и серверной части приложения, предоставляющего API и бизнес-логику. Пользовательский интерфейс планируется реализовать с помощью метода отрисовки веб-страницы, при котором HTML-код генерируется на сервере.

Под проектированием понимается визуализация структуры данных, связей сущностей, интерфейсов и сценариев взаимодействия с внешними системами до начала или параллельно с реализацией. Перед физическим проектированием схемы определяются СУБД, в которой будут храниться учетные данные. Критерии: поддержка транзакций, совместимость с серверным стеком, пригодность для продуктивной эксплуатации и тестового контура.

Для хранения данных в продуктивной среде обычно выбирают полноценную клиент–серверную СУБД. Рассмотрим следующие СУБД (таблица 1).

- MySQL – клиент–серверная СУБД с широким распространением в веб-проектах: отдельный процесс сервера, параллельные подключения, SQL, транзакции (InnoDB), резервное копирование и администрирование.
- PostgreSQL – объектно-реляционная СУБД с расширенными типами данных (JSON, массивы, полнотекстовый поиск), строгим соответствием

стандартам SQL, развитыми механизмами целостности и репликации; часто выбирается для сложной аналитики и крупных корпоративных систем.

– SQLite – встраиваемая СУБД: база в одном файле, без отдельного серверного процесса; удобна для локальной разработки и автоматизированных тестов.

Таблица 1 – Сравнение средств реализации базы данных

Критерий	MySQL	PostgreSQL	SQLite
Богатый набор типов и возможностей серверной СУБД	+	+	–
Распространённость при реализации веб-приложений	+	+	±
Транзакции и целостность данных	+	+	+
Работа без отдельного сервера БД	–	–	+
Документация и сообщество разработчиков	+	+	+
Соответствие стеку проекта (Laravel, типовой хостинг)	+	+	±

В качестве основной СУБД для продуктивной среды выбрана MySQL;

Инструменты визуального проектирования схемы CASE (таблица 2):

– MySQL Workbench – официальное средство моделирования и администрирования MySQL: ER-диаграммы, генерация DDL, обратный инжиниринг.

– phpMyAdmin – веб-клиент к MySQL: просмотр структуры, выполнение SQL, импорт/экспорт; слабее для полноценного ER-моделирования, но удобен на хостинге.

Таблица 2 – Сравнение средств проектирования БД

Критерий	MySQL Workbench	phpMyAdmin
Визуальное ER-моделирование	+	–
Прямой перенос схемы на сервер MySQL	+	±
Обратный инжиниринг существующей БД	+	±
Доступность без установки на ПК разработчика	–	+
Согласование с миграциями Laravel	±	±

Для разработки и согласования ER-модели целесообразно использовать MySQL Workbench, а для операций с данными и структурой на сервере – phpMyAdmin. Физическая схема в репозитории фиксируется миграциями Laravel (database/migrations/), поэтому проектирование должно совпадать с принятыми правилами именования таблиц и полей в команде.

Помимо схемы БД документируют потоки данных, взаимодействие с внешними системами (API, очереди, вебхуки), роли подсистем.

Рассмотренные средства (таблица 3):

- Draw.io (diagrams.net) – универсальный редактор в браузере или десктопе: блок-схемы, DFD, схемы интеграций, нотации IDEF0/BPMN в упрощённом виде.
- PlantUML – текстовое описание диаграмм (последовательности, компонентов); удобно хранить рядом с кодом в репозитории.
- Miro – облачная доска для совместной проработки процессов с заказчиком; слабее для формальной технической спецификации.

Таблица 3 – Сравнение средств проектирования диаграмм

Критерий	Draw.io	PlantUML	Miro
Диаграммы процессов и интеграций без привязки к СУБД	+	+	+
Включение в отчёт (экспорт PNG/PDF/SVG)	+	+	+
Версионирование в Git (текстовый исходник)	±	+	–
Низкий порог входа для нетехнических участников	+	–	+
Офлайн-работа	+	+	–

В качестве средства проектирования диаграмм было выбрано Draw.io.

На этапе проектирования UI фиксируют структуру экранов, навигацию, ключевые элементы форм и согласуют их с ролями пользователей до или параллельно с версткой в blade-шаблонах.

Рассмотренные средства (таблица 4):

- Figma – облачное прототипирование: макеты, компоненты, комментарии заказчика.

- Draw.io – упрощённые wireframe-схемы экранов и переходов; не заменяет UI-kit, но достаточен для отчёта.

Таблица 4 – Сравнение средств проектирования интерфейсов

Критерий	Figma	Draw.io
Детальные макеты и визуальный стиль	+	–
Согласование с заказчиком	+	+
Связь с реальной реализацией проекта	–	–
Стоимость и доступность	±	+

Для проектирования прототипов пользовательского интерфейса был выбран Draw.io.

Выбор языка программирования определяет экосистему фреймворков, доступ к СУБД, модель развёртывания и квалификацию команды. Для данного проекта серверная логика и клиентская интерактивность разделены: PHP на сервере, JavaScript в браузере.

Краткая характеристика кандидатов для серверной части (таблица 5):

- PHP – язык, ориентированный на веб и HTTP; десятилетия применения с MySQL; в проекте используется версия ^8.2.

- Python – универсальный язык с сильными фреймворками Django и FastAPI; выше порог организации типового LAMP-стека для команды, привыкшей к PHP-хостингу.

- JavaScript (Node.js) – единый язык на сервере и клиенте; асинхронная модель и отдельный рантайм усложняют сопровождение при уже принятом PHP-контуре данных.

Таблица 5 – Сравнение языков для серверной разработки

Критерий	PHP	Python	Node.js
Зрелая экосистема для веб и типового хостинга	+	+	+
Прямая интеграция с MySQL в учебном/корпоративном контуре	+	+	+
Наличие «батареек» (ORM, миграции, очереди) в одном стеке	+	±	±
Соответствие текущему проекту	+	–	–

Для серверной части выбран PHP 8.2. Клиентская логика интерфейса реализуется на JavaScript в составе страниц, сформированных сервером, с разметкой и стилями – HTML и CSS (Tailwind).

Фреймворк задаёт архитектуру HTTP-приложения: маршрутизация, слой доступа к данным, валидация, аутентификация, фоновые задачи, тестирование.

Рассмотренные PHP-фреймворки (таблица 6):

- Laravel – полнофункциональный фреймворк: Eloquent ORM, миграции, очереди, политики, Sanctum, встроенная интеграция с PHPUnit; в проекте – Laravel 12.
- Yii – акцент на производительность и явные паттерны MVC; сильная типизация конфигурации, меньше «соглашений по умолчанию», чем у Laravel.
- Symfony (как альтернатива уровня enterprise) – модульные компоненты, гибкость, более высокая начальная сложность сборки приложения.

Таблица 6 – Сравнение PHP-фреймворков

Критерий	Laravel	Yii	Symfony
ORM и миграции схемы	+	+	+
Аутентификация, политики, API	+	+	±
Очереди и фоновые задачи	+	+	+
Документация и экосистема пакетов	+	+	+

Критерий	Laravel	Yii	Symfony
Скорость старта учётного веб-приложения	+	±	–

Для реализации веб-приложения выбран серверный фреймворк Laravel.

Интерфейс построен как классическое веб-приложение с перезагрузкой страницы и серверными формами. Данные передаются при запросе страницы и отправке форм.

– Blade – используется для построения классического серверного веб-интерфейса: страницы формируются на стороне Laravel, а для разных ролей пользователей могут применяться отдельные шаблоны и макеты. Такой подход соответствует фактической реализации проекта, упрощает работу с сессиями, формами и серверной валидацией.

– Vue 3 – фреймворк для создания компонентных одностраничных приложений, также удобен для сложных интерактивных интерфейсов, однако требует отдельной организации клиентской маршрутизации, состояния приложения и сборки фронтенда.

– React – используется для создания компонентных пользовательских интерфейсов и одностраничных приложений, обладает развитой экосистемой, но для данного проекта является избыточным, так как основная логика интерфейса реализуется через серверные страницы и формы.

Сравнение вариантов фреймворков клиентской части представлено в таблице 7.

Таблица 7 – Сравнение вариантов клиентской части

Критерий	Blade	Vue 3	React
Прогрессивное внедрение в legacy	+	+	-
Готовые UI компоненты	±	+	+
Скорость разработки форм	+	±	±
Переиспользование UI-компонентов	-	+	+
Подходит для лендинга или простого веб-приложения	+	±	-
Удобство поддержки сложной клиентской логики	-	+	+
Необходимость отдельной сборки фронтенда	-	+	+

Для клиентской части выбран стек Blade.

Среда разработки исходного кода (таблица 8):

- Visual Studio Code – редактор с расширениями для PHP, Laravel и Git; бесплатный, привычный интерфейс.
- PHPStorm – специализированная IDE для PHP с развитой отладкой; платная лицензия.

Таблица 8 – Сравнение сред разработки

Критерий	VS Code	PHPStorm
Отладка PHP	+	+
Расширения и гибкость	+	–
Стоимость для учебного проекта	+	–
Соответствие выбору разработчика	+	±

После анализа для разработки выбран Visual Studio Code.

Тестирование охватывает автоматическую проверку кода (регрессия), ручную проверку сценариев по ролям и при необходимости нагрузочные пробы.

Рассмотренные средства (таблица 9):

- PHPUnit – стандарт автотестов PHP в Laravel: feature-тесты (tests/Feature/), прогон `php artisan test`; в проекте PHPUnit 11, тесты модуля студентов и др.
- Postman – коллекции HTTP-запросов для ручной проверки маршрутов и API (вспомогательно).
- k6 – нагрузочное тестирование сценариев входа, гостевых страниц и разделов под нагрузкой (load/, скрипты login-load.js, guest-load.js и др.).

Таблица 9 – Сравнение средств тестирования

Критерий	PHPUnit	Postman	k6
Автоматические регрессионные тесты кода	+	–	–
Проверка HTTP API	+	+	±
Встраивание в CI повторяемый прогон	+	±	+
Нагрузочное тестирование	–	–	+
Роль в проекте	основное	вспомогательное	по необходимости

Основное средство автоматического тестирования – PHPUnit. k6 применяется для оценки устойчивости отдельных сценариев под нагрузкой. Postman может использоваться для отладки и приёма HTTP-запросов.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист
						16
Изм.	Лист	№ докум.	Подпись	Дата		

2 Техническое задание на разработку программного продукта

В начале разработки создавалось техническое задание, в котором указывались основные требования.

Для создания технического задания использовался стандарт ГОСТ 34.602-2020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

Согласно ГОСТ 34.602-2020 техническое задание должно включать следующие разделы:

1. Введение.
2. Основания для разработки.
3. Назначение системы.
4. Требования к системе.
5. Требования к техническому обеспечению.
6. Требования к программному обеспечению.
7. Требования к тестированию.
8. Организационно-технические требования.

Техническое задание на разработку веб-приложения представлено в приложении А.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 17
Изм.	Лист	№ докум.	Подпись	Дата		

3 Проектирование программного продукта

3.1 Архитектура программного продукта

Веб-приложение спортивного клуба «Авиатор» относится к классу внутренних учётных информационных систем и предназначено для автоматизации учёта студентов и команд, видов спорта, соревнований и тренировок, публикации новостей, приёма заявок на вступление в команды и участие в соревнованиях, а также для отображения публичной информации неавторизованным пользователям.

Архитектура построена по клиент–серверному принципу. Клиентская часть работает в браузере: отображение HTML-страниц, навигация по разделам, отправка форм (POST) и точечные запросы JavaScript. Серверная часть обрабатывает HTTP-запросы, выполняет бизнес-логику, проверяет права доступа, работает с MySQL и внешними системами.

Серверная часть реализована на PHP 8.2 с фреймворком Laravel 12: маршрутизация (routes/web.php), контроллеры, Eloquent ORM, валидация, сессионная аутентификация, защита от CSRF, шаблоны Blade.

Клиентская часть формируется на сервере (Blade), оформление – HTML и CSS (Tailwind CSS), для сборки минимизированных компонентов Javascript и CSS используется Vite. Обмен с сервером – HTTP (перезагрузка страниц и отправка форм), а не единый JSON-API для всего приложения.

Данные хранятся в MySQL: пользователи, виды спорта, команды, составы, соревнования, результаты, тренировки, записи на тренировки, новости, заявки в команды и на соревнования, справочники локаций и др.

Разграничение доступа – на сервере: после входа определяется роль «студент» или «преподаватель».

Внешние взаимодействия: аутентификация через Active Directory (LDAP); загрузка и разбор расписания учебных групп с сайта расписания техникума, уведомления подписчиков в боте Макс.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 18
Изм.	Лист	№ докум.	Подпись	Дата		

Используется архитектурный шаблон MVC: модели Eloquent – БД; контроллеры – запросы и логика; представления – Blade (отдельные layout для гостя, студента, преподавателя).

На рисунке 1 диаграмма компонентов: клиент, сервер (маршруты, middleware, контроллеры, сервисы, модели), MySQL, внешние системы.

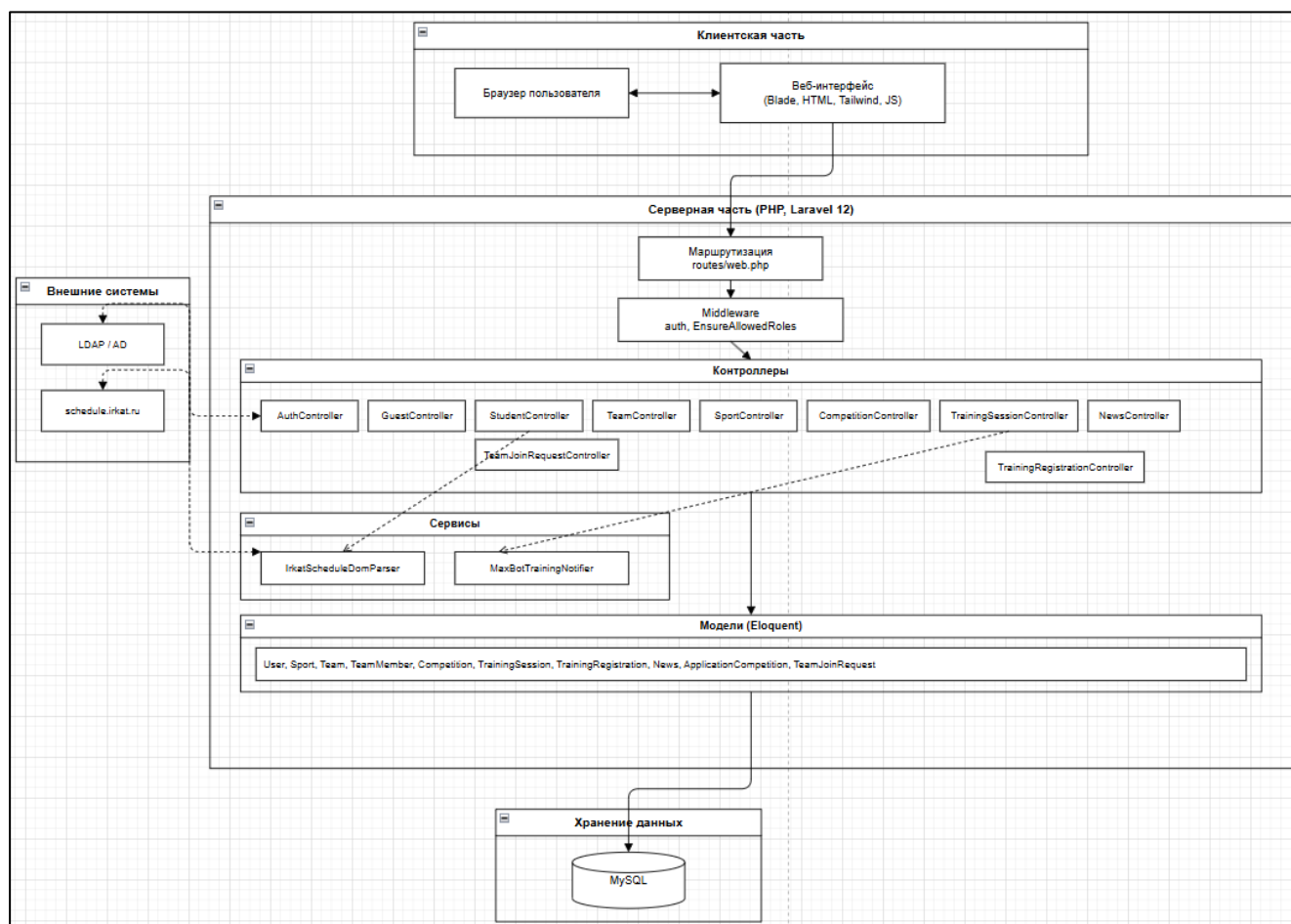


Рисунок 1 – Диаграмма компонентов

3.2 Функциональное и структурное проектирование

Для наглядного отображения функциональности и логики работы веб-приложения спортивного клуба «Авиатор» были разработаны диаграммы, позволяющие описать взаимодействие пользователей с системой и последовательность выполнения основных процессов. В рамках проектирования

были рассмотрены ключевые сценарии работы приложения: просмотр информации о спортивном клубе, подача заявок, вступление студентов в команды, создание тренировок и запись студентов на тренировочные занятия.

На рисунке 2 представлена диаграмма прецедентов, которая отображает взаимодействие пользователей с веб-приложением спортивного клуба «Авиатор». На диаграмме отражены основные роли системы: неавторизованный пользователь, студент и преподаватель. Неавторизованный пользователь может просматривать публичную информацию о спортивном клубе. Студент получает доступ к личным функциям после авторизации: просмотру тренировок, соревнований, команд, подаче заявок и просмотру истории участия. Преподаватель выполняет функции управления спортивной информацией: создаёт и редактирует тренировки, соревнования, команды, новости, а также рассматривает заявки студентов.

Неавторизованный пользователь может просматривать публичную информацию о спортивном клубе: расписание работы, контакты, общие новости, перечень секций и требования для вступления. Для него недоступны действия, требующие идентификации.

Студент получает доступ к личным функциям после авторизации (вход по логину и паролю, полученному через учебное заведение). Ему доступны: просмотр расписания тренировок и предстоящих соревнований, просмотр состава команд клуба, подача заявок на вступление в выбранную секцию или команду, просмотр истории участия в тренировках и соревнованиях, а также запись на тренировочные занятия с возможностью отмены за определенное время.

Преподаватель выполняет функции управления спортивной информацией: создаёт, редактирует и отменяет тренировки и соревнования, формирует и корректирует составы команд, публикует и удаляет новости, а также рассматривает входящие заявки студентов — подтверждает или отклоняет их с обязательным указанием причины отказа.

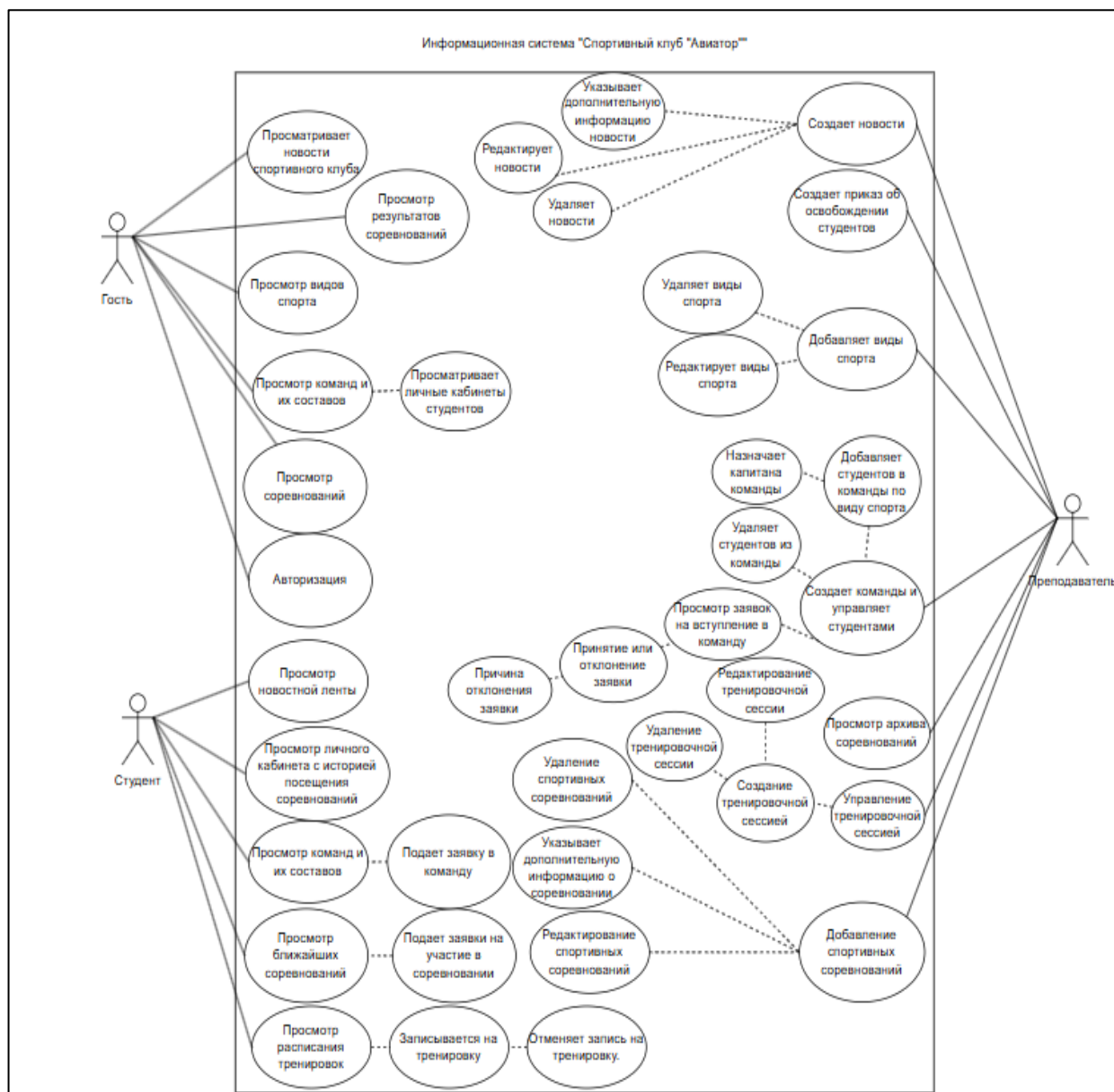


Рисунок 2 – Диаграмма прецедентов

На рисунке 3 представлена контекстная диаграмма IDEF0, отображающая работу веб-приложения.

На контекстной диаграмме IDEF0 работа веб-приложения спортивного клуба «Авиатор» представлена как преобразование входящих заявок и условий записи в команды, расписания, результаты соревнований, обработанные заявки и отчётные данные. Управление процессом осуществляется на основе прав доступа, правил клуба и регламентов мероприятий. Механизмами выполнения выступают студент и преподаватель.

Изм.	Лист	№ докум.	Подпись	Дата

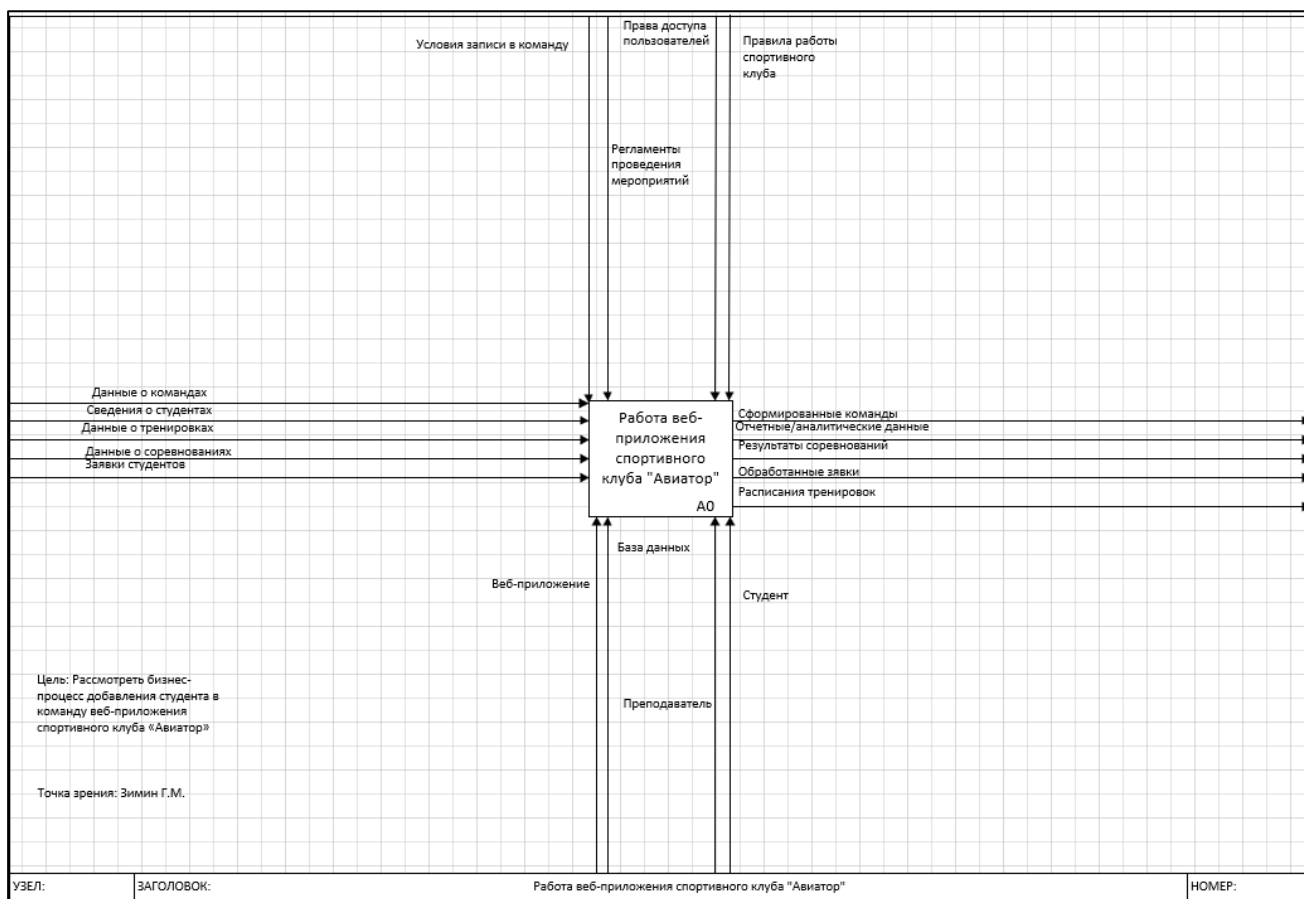


Рисунок 3 – Контекстная диаграмма IDEF0

На рисунке 4 представлена диаграмма декомпозиции (A1), раскрывающая процесс добавления студентов в группу. Данная диаграмма детализирует логику включения пользователя в состав команды или секции клуба «Авиатор» на основе его заявки и проверки всех необходимых условий.

Процесс A1 разбит на ряд взаимосвязанных подпроцессов и потоков данных, которые используют следующие информационные объекты системы: «Условия записи в команду», «Права доступа пользователя», «Регламент проведения мероприятий», «Сведения о пользователях», результаты «Авторизации в веб-приложении», данные о «Заявках студентов», «Данные о командах», «Данные о соревнованиях» и «Данные о тренировках», а также общую структуру «Веб-приложения».

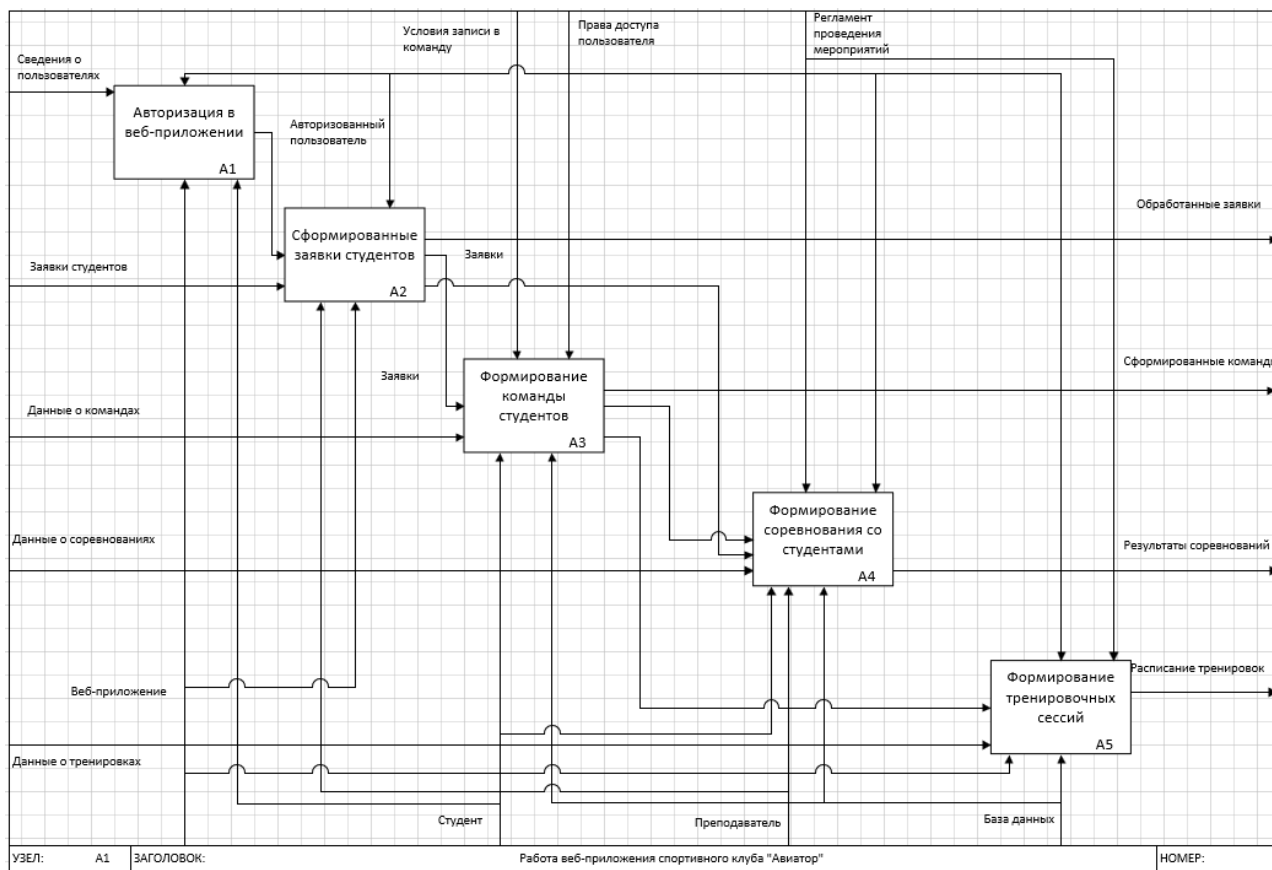


Рисунок 4 – Диаграмма декомпозиции(A1)

На рисунке 5 представлена диаграмма деятельности, отражающая процесс работы преподавателя и студента с тренировками. Процесс начинается с авторизации преподавателя в системе. После входа преподаватель переходит в раздел тренировок, выбирает действие создания тренировочной сессии, заполняет форму и сохраняет тренировку. После создания система присваивает тренировке статус «Запланирована».

После этого студент авторизуется в веб-приложении, переходит в раздел тренировок и просматривает доступные тренировочные сессии. Если тренировка находится в статусе «Запланирована», студент может записаться на неё. Система сохраняет запись студента на тренировку и отображает результат выполненного действия.

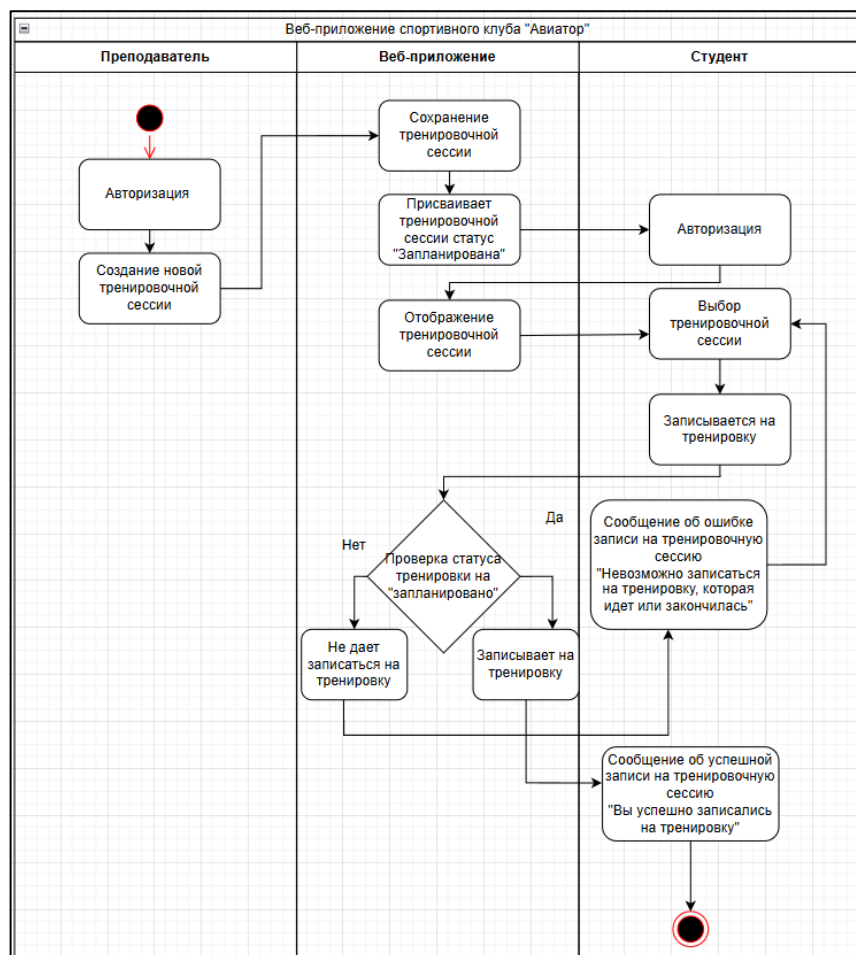


Рисунок 5 – Диаграмма деятельности

Таким образом, функциональное и структурное проектирование позволило определить основные роли пользователей, ключевые сценарии взаимодействия с веб-приложением и последовательность выполнения операций. Построенные диаграммы отражают работу с командами, тренировками и заявками студентов, что служит основой для дальнейшего проектирования базы данных, серверной части и пользовательского интерфейса.

3.3 Проектирование базы данных

ER-модель (Entity–Relationship, модель «сущность–связь») – это способ концептуального моделирования данных, который описывает сущности, их атрибуты и связи между ними. Она используется для проектирования баз данных и

систем, где важно понять структуру и взаимосвязь информации до начала разработки.

На рисунке 6 представлена ER-диаграмма базы данных.

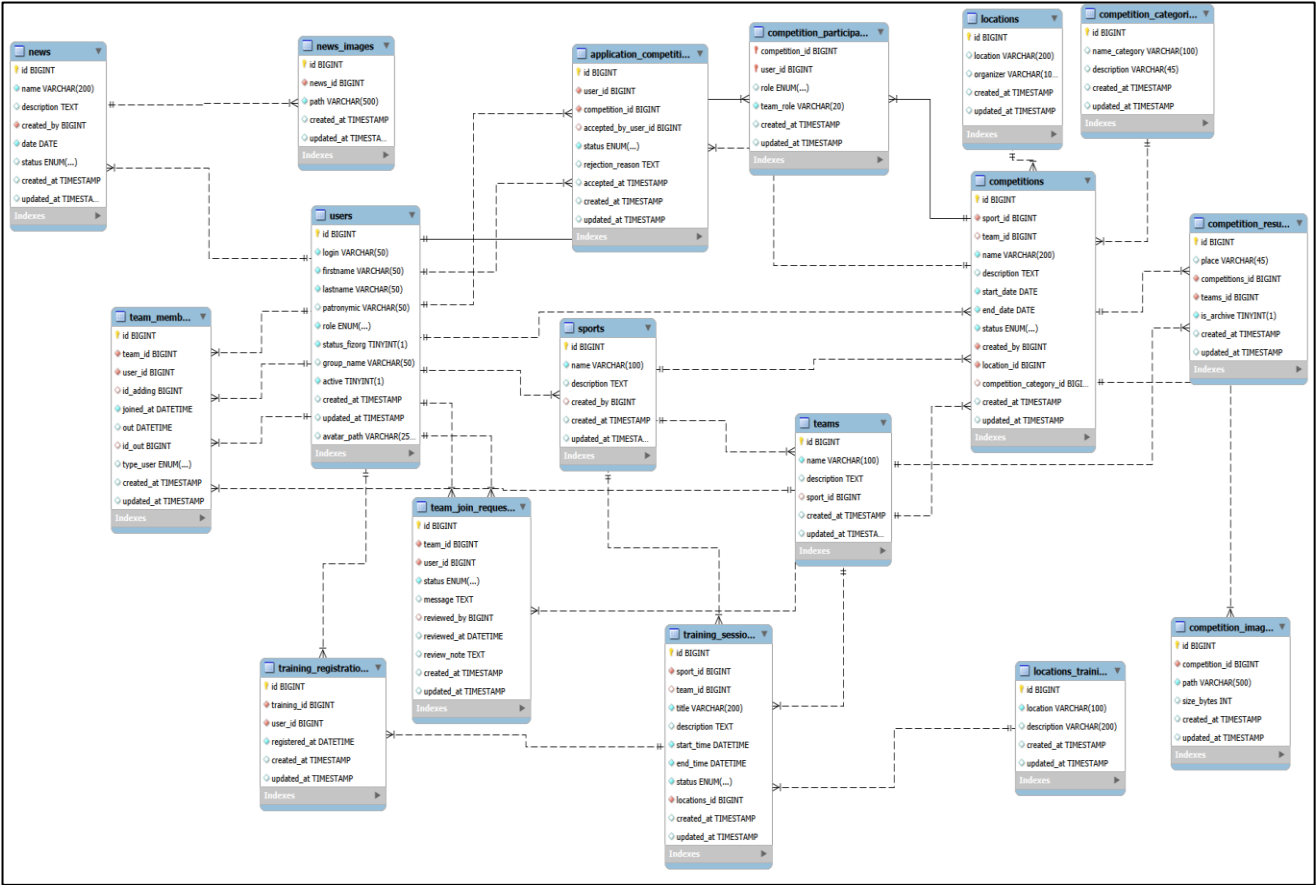


Рисунок 6 – ER-модель

В ходе проектирования базы данных были выделены основные сущности предметной области и связи между ними. Повторяющиеся данные вынесены в отдельные таблицы: пользователи, виды спорта, команды, соревнования, тренировки, локации и категории соревнований. Большинство таблиц приведено к третьей нормальной форме, так как неключевые атрибуты зависят от первичного ключа и не содержат транзитивных зависимостей. Поля типа ENUM используются для хранения ограниченного набора статусов и ролей пользователей.

Подробное содержание таблиц представлено в таблицах 10–25.

Таблица 10 – Таблица «application_competition»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ заявки
user_id	BIGINT(20)	Внешний ключ таблицы «Users»
competition_id	BIGINT(20)	Внешний ключ таблицы competition
accepted_by_user_id	BIGINT(20)	Внешний ключ таблицы «users»
status	ENUM («pending», «accepted», «rejected»)	Статус заявки («ожидает рассмотрения», «принято», «отклонено»)
rejection_reason	TEXT	Причина отклонения
accepted_at	TIMESTAMP	Дата и время принятия заявки
created_at	TIMESTAMP	Дата и время создания заявки
updated_at	TIMESTAMP	Дата и время обновления заявки

Таблица 11 – Таблица «competitions»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
sport_id	BIGINT(20)	Внешний ключ таблицы «sports»
team_id	BIGINT(20)	Внешний ключ таблицы «teams»
name	VARCHAR(200)	Название соревнования
description	TEXT	Описание соревнования
start_date	DATE	Дата начала
end_date	DATE	Дата окончания
status	ENUM («upcoming», «ongoing», «finished», «cancelled»)	Статус соревнования («предстоящий», «текущий», «завершенный», «отмененный»)
created_by	BIGINT(20)	Внешний ключ на «users»
location_id	BIGINT(20)	Внешний ключ таблицы «locations»
competition_category_id	BIGINT(20)	Внешний ключ таблицы «Competition_categories»
created_at	TIMESTAMP	Дата и время создания соревнования
updated_at	TIMESTAMP	Дата и время обновления соревнования

Таблица 12 – Таблица «competition_categories»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
name_category	VARCHAR(100)	Название категории
description	VARCHAR(45)	Описание категории
created_at	TIMESTAMP	Дата и время создания заявки
updated_at	TIMESTAMP	Дата и время обновления заявки

Таблица 13 – Таблица «competition_images»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
competition_id	BIGINT(20)	Внешний ключ таблицы «competition»
path	VARCHAR(500)	Путь к фотографиям
size_bytes	INT(10)	Размер фотографии
created_at	TIMESTAMP	Дата и время добавления фотографии
updated_at	TIMESTAMP	Дата и время обновления фотографии

Таблица 14 – Таблица «competition_participants»

Поле	Тип данных	Описание поля
competition_id	BIGINT(20)	Внешний ключ таблицы «competitions»
user_id	BIGINT(20)	Внешний ключ таблицы «users»
role	ENUM («student», «teacher»)	Роль участника в соревновании («студент», «преподаватель»)
team_role	VARCHAR(20)	Роль в соревновании
created_at	TIMESTAMP	Дата и время вступления на участие в соревновании
updated_at	TIMESTAMP	Дата и время обновления на участие в соревновании

Таблица 15 – Таблица «competition_results»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
place	VARCHAR(45)	Место, занятое в соревновании
competitions_id	BIGINT(20)	Внешний ключ таблицы «competitions»
teams_id	BIGINT(20)	Внешний ключ таблицы «teams»
is_archive	TINYINT(1)	Статус архива (1, 0)
created_at	TIMESTAMP	Дата и время создания результата
updated_at	TIMESTAMP	Дата и время редактирования результата

Таблица 16 – Таблица «locations»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
location	VARCHAR(200)	Название локации
organizer	VARCHAR(100)	Организатор соревнования
created_at	TIMESTAMP	Дата и время создания локации
updated_at	TIMESTAMP	Дата и время редактирования локации

Таблица 17 – Таблица «locations_training»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
location	VARCHAR(100)	Название локации
description	VARCHAR(200)	Описание локации
created_at	TIMESTAMP	Дата и время создания локации
updated_at	TIMESTAMP	Дата и время редактирования локации

Таблица 18 – Таблица «news»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
name	VARCHAR(200)	Название новости
description	TEXT	Описание новости
created_by	BIGINT(20)	Внешний ключ таблицы «users», создатель новости
date	DATE	Дата публикации
status	ENUM («Published», «Draft»)	Статус новости («Опубликовано», «Черновик»)
created_at	TIMESTAMP	Дата и время создания новости
updated_at	TIMESTAMP	Дата и время редактирования новости

Таблица 19 – Таблица «news_images»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
news_id	BIGINT(20)	Внешний ключ таблицы «news»
path	VARCHAR(500)	Путь к фотографии
created_at	TIMESTAMP	Дата и время создания фотографии новости
updated_at	TIMESTAMP	Дата и время редактирования фотографии новости

Таблица 20 – Таблица «sports»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
name	VARCHAR(100)	Название спорта
description	TEXT	Описание спорта
created_by	BIGINT(20)	Внешний ключ таблицы «users»
created_at	TIMESTAMP	Дата и время создания спорта
updated_at	TIMESTAMP	Дата и время изменения спорта

Таблица 21 – Таблица «teams»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
name	VARCHAR(100)	Название команды
description	TEXT	Описание команды
sport_id	BIGINT(20)	Внешний ключ таблицы «sports»
created_at	TIMESTAMP	Дата и время создания команды
updated_at	TIMESTAMP	Дата и время редактирование команды

Таблица 22 – Таблица «team_member»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
team_id	BIGINT(20)	Внешний ключ таблицы «teams»
user_id	BIGINT(20)	Внешний ключ таблицы «users», студент
id_adding	BIGINT(20)	Внешний ключ таблицы «users», преподаватель
joined_at	DATETIME	Дата и время вступления в команду
out	DATETIME	Дата и время выхода из команды
type_user	ENUM(«coach», «capitan», «member»)	Статус студента в команде («тренер», «капитан», «участник»)
created_at	TIMESTAMP	Дата и время вступления
updated_at	TIMESTAMP	Дата и время обновления

Таблица 23 – Таблица «training_registrations»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
training_id	BIGINT(20)	Внешний ключ таблицы «trainings»
user_id	BIGINT(20)	Внешний ключ таблицы «users»
registered_at	DATETIME	Дата и время регистрации
created_at	TIMESTAMP	Дата и время создания
updated_at	TIMESTAMP	Дата и время обновления

Таблица 24 – Таблица «training_sessions»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
sport_id	BIGINT(20)	Внешний ключ таблицы «sports»
team_id	BIGINT(20)	Внешний ключ таблицы «teams»
title	VARCHAR(200)	Название тренировки
description	TEXT	Описание тренировки
start_time	DATETIME	Дата и время начала тренировки
end_time	DATETIME	Дата и время окончания тренировки

Продолжение таблицы 24

status	ENUM(«scheduled», «in_progress», «cancelled», «completed»)	Статус тренировки(«запланировано», «в процессе», «отменено», «завершено»)
locations_id	BIGINT(20)	Внешний ключ таблицы «locations»
created_at	TIMESTAMP	Дата и время создания
updated_at	TIMESTAMP	Дата и время обновления

Таблица 25 – Таблица «users»

Поле	Тип данных	Описание поля
id	BIGINT(20)	Первичный ключ
login	VARCHAR(50)	Логин пользователя
firstname	VARCHAR(50)	Имя пользователя
lastname	VARCHAR(50)	Фамилия пользователя
patronymic	VARCHAR(50)	Отчество пользователя
role	ENUM(«student», «teacher»)	Роль пользователя(«студент», «преподаватель»)
status_fizorg	TINYINT(1)	Статус физического организатора (1, 0)
group_name	VARCHAR(50)	Название группы
active	TINYINT(1)	Активность учетной записи LDAP(1, 0)
created_at	TIMESTAMP	Дата и время создания пользователя
updated_at	TIMESTAMP	Дата и время обновления данных
avatar_path	VARCHAR(255)	Путь к фотографии пользователя

Таким образом, спроектированная структура базы данных обеспечивает хранение сведений о пользователях, спортивных дисциплинах, командах, тренировках, соревнованиях, заявках и результатах. Связи между таблицами позволяют учитывать участие студентов в командах, регистрацию на тренировки, подачу заявок на соревнования и фиксацию спортивных результатов.

3.4 Проектирование пользовательского интерфейса

В разделе представлены прототипы основных страниц, отражающие структуру интерфейса и расположение функциональных элементов. Также приведены макеты отдельных страниц, уточняющие визуальное представление интерфейса: цветовые акценты, расположение блоков, карточек, таблиц и элементов навигации.

Проектирование пользовательского интерфейса выполнялось с учётом ролей пользователей веб-приложения: неавторизованный пользователь, студент и преподаватель. Для каждой роли были определены доступные разделы, основные пользовательские действия и состав отображаемой информации.

При проектировании интерфейса учитывались следующие требования: понятная навигация, разделение функций по ролям, наличие форм ввода данных, отображение списков с фильтрацией и пагинацией, а также предоставление пользователю сообщений о результате выполненных действий.

Для разработки прототипов пользовательского интерфейса использовался инструмент Draw.io. Прототипы позволяют определить расположение основных элементов страниц, структуру навигации и последовательность действий пользователя до этапа реализации программного продукта.

На рисунке 7 представлен прототип главной страницы для роли «Неавторизованный пользователь». В верхней части страницы расположена навигационная полоса с логотипом «Авиатор» и пунктами меню: «Главная», «Новости», «Виды спорта», «Команды», «Результаты соревнований», а также кнопка «Войти»; на узких экранах предусмотрено сворачивание меню в мобильную панель.

Основная рабочая зона страницы организована вертикальным набором из пяти секций, каждая из которых имеет заголовок, ссылку перехода ко всему разделу («Все новости →», «Все соревнования →») и сетку из четырёх карточек:

1. «Последние новости» – карточка с заголовком, кратким текстом, датой и кнопкой «Подробнее»; переход на экран новости.
2. «Предстоящие соревнования» – название, статус «Предстоящее», вид спорта, даты, место проведения, кнопка «Подробнее».
3. «Результаты соревнований» – название соревнования, занятое место, описание или вид спорта, дата, кнопка «Подробнее».
4. «Виды спорта» – название, краткое описание, кнопка «Подробнее».

5. «Команды» – название, вид спорта, число участников, кнопка «Подробнее».

Логотип	Главная	Новости	Спорт	Команды	Результаты	[?]	Войти
---------	----------------	---------	-------	---------	------------	-----	-------

Последние новости

Все новости →

Заголовок Текст... дд.мм.гг [Подробнее]	Карточка	Карточка	Карточка
--	----------	----------	----------

[пусто: состояние: нет новостей]

[<] (1) (2) [>]

Предстоящие соревнования

Все соревнования →

Заголовок [статус] — вид спорта — даты — место [Подробнее]	Карточка	Карточка	Карточка
---	----------	----------	----------

Результаты соревнований

Все результаты →

Заголовок [Место: N] Текст... Дата [Подробнее]	Карточка	Карточка	Карточка
--	----------	----------	----------

Виды спорта

Все виды спорта →

Название Описание... [Подробнее]	Карточка	Карточка	Карточка
--	----------	----------	----------

Команды

Все команды →

Название Вид спорта Описание... Участников: N [Подробнее]	Карточка	Карточка	Карточка
---	----------	----------	----------

© Копирайт

Рисунок 7 – Прототип главной страницы «Гостя»

На рисунке 8 представлен прототип страницы авторизации.

В форме размещены поля «Логин» и «пароль», а также кнопка «Войти».

При ошибке входа над формой показывается уведомление с текстом причины (например, «Вход запрещен», «Ваша учетная запись заблокирована», «Неверный логин или пароль»).

При успешной проверке данных пользователь автоматически входит в систему и перейдет на главную страницу в зависимости от роли пользователя. При неуспешной авторизации, отображается сообщение об ошибке.

Авторизация

Логин
Введите ваш логин

Пароль
Введите ваш пароль

[Войти]

Рисунок 8 – Прототип страницы авторизации

На рисунке 9 представлен прототип страницы «Тренировочные сессии» для роли студент. Слева – боковое меню панели студента.

В основной области находится заголовок «Тренировочные сессии», панель фильтров (вид спорта, период дат, поиск по названию) и переключатель отображения «Список/Карточки». Ниже фильтр по статусу: «Все», «Запланированные», «Идут сейчас», «Завершенные», «Отмененные».

Тренировки отображаются карточками или таблицей в режиме списка. На карточке запланированной тренировки – кнопки «Записаться», а также «Подробнее». Внизу находится пагинация.

Тренировочные сессии

[Вид спорта ▼] [Дата с] [Дата по] [Поиск...] [Найти] [Сброс]

[\[Список | Карточки \]](#)

Статус: (Все) (Запланированные) (Идут сейчас) (Завершенные) (Отмененные)

Название [Запланирована] вид спорта дд.мм.гггг, чч.мм – чч.мм локация краткое описание... [Записаться] [Подробнее]	Название [Запланирована] ... [Записан] [Подробнее]	Карточка 3 [Подробнее]	Карточка 4
---	--	---	---

Рисунок 9 – Прототип страницы «Тренировочные сессии»

На рисунке 10 представлен прототип страницы «Соревнования» для роли студент.

Слева находится панель студента. В основной области – заголовок «Соревнования» и панель фильтров: Вид спорта, период дат, поиск по названию. Ниже фильтр по статусу: «Все», «Предстоящие», «Идут сейчас», «Завершенные».

Панель студента

Главная

Профиль

Соревнования

Результаты соревнований

Тренировки

Новости

Команды

Выйти

Соревнования

[Вид спорта ▼] [Дата с] [Дата по] [Поиск по названию...] [Найти] [Сброс]

Статус: (Все) (Предстоящие) (Идут сейчас) (Завершенные)

[обложка]

Название [Предстоящее]

вид спорта

дд.мм.гггг — дд.мм.гггг

локация

описание...

[Подробнее]

Карточка 2

[Идет]

[Подробнее]

Карточка 3

[Завершено]

Карточка 4

[◀] 1 2 3 [▶]

Рисунок 10 – Прототип страницы «Соревнования»

На рисунке 11 представлен прототип страницы «Профиль».

В центре карточка профиля. В шапке – аватар, заголовок «Личный кабинет» и кнопка «Изменить фотографию».

Ниже блок «Основная информация»: логин, ФИО, группа. Блок «Дополнительная информация» - статус физического организатора.

Блок «Статистика» - три карточки со счетчиками: «Команды», «Соревнования», «Тренировки».

Панель студента Главная Профиль Соревнования Результаты Тренировки Новости Команды [Выйти]	() аватар Личный кабинет [Изменить фотографию]		
	Основная информация Логин [значение] Имя [значение] Фамилия [значение] Отчество [значение] Группа [значение]		
	Дополнительная информация Статус физорга [Да / Нет]		
	Статистика Команды N → история Соревнования N → история Тренировки N → история		

Рисунок 11 – прототип страницы «Профиль»

На рисунке 12 представлен макет страницы авторизации. Фон страницы – светло-серый, а по центру – белая карточка.

Внутри карточки, по центру – заголовок «Авторизация» тёмно-серым крупным полужирным шрифтом. При ошибке входа над полями – зона уведомления со светло-розовым фоном и красным текстом причины.

Ниже – поля формы с подписями «Логин» и «Пароль»: подписи тёмно-серые, поля на всю ширину карточки с белым фоном и светло-серой обводкой, внутри – серые подсказки «Введите ваш логин» и «Введите ваш пароль».

Кнопка «Войти» на всю ширину формы с белым текстом и синей заливкой, скруглённые углы.

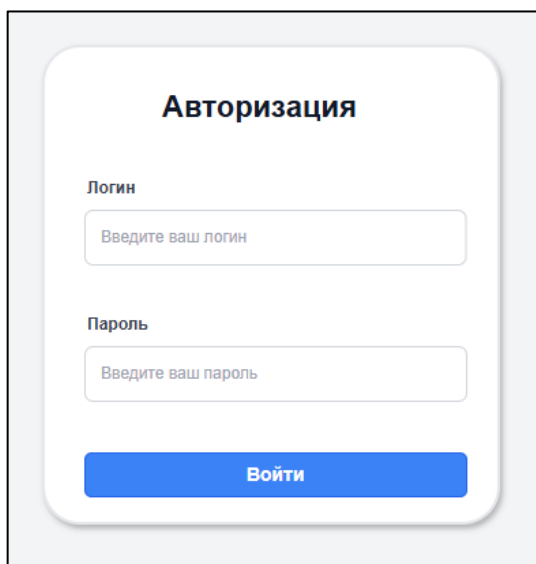


Рисунок 12 – Макет страницы авторизации

На рисунке 13 представлен макет главной страницы пользователя с ролью «Преподаватель».

Фон рабочей области – светло-серый. Слева – белая боковая панель «Панель преподавателя» с пунктами меню; активен «Главная» (синий акцент). Внизу панели – красная кнопка «Выйти».

Справа – основное содержимое из трёх блоков.

«Последние новости» – заголовок тёмно-серым крупным шрифтом, справа ссылка «Все новости» синим. Ниже сетка из трёх белых карточек со скруглением и тенью: сверху цветная полоса-обложка, заголовок, краткий текст, дата, синяя кнопка «Подробнее» с белым текстом.

«Ближайшие соревнования» – заголовок и две ссылки: «Результаты» зелёным и «Все соревнования» синим. Карточки соревнований в том же стиле: название, голубая метка статуса («Предстоящее»), вид спорта, даты, место, кнопка «Подробнее».

«Ближайшие тренировки» – заголовок и ссылка «Все тренировки» синим. Три карточки тренировок с названием, статусом, датой и местом, кнопка «Подробнее» на каждой.

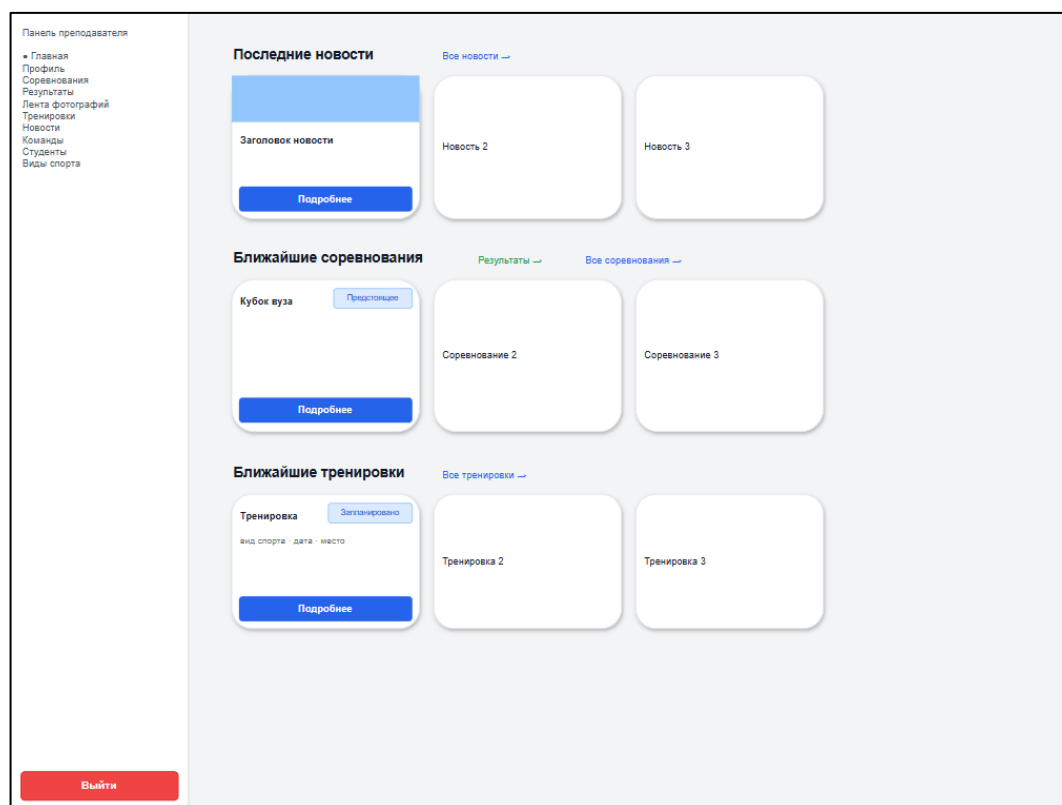


Рисунок 13 – Макет главной страницы преподавателя

На рисунке 14 представлена страница «Соревнования» в макете для преподавателя. Фон – светло-серый. Слева – белая боковая панель, активен пункт «Соревнования», внизу – красная кнопка «Выйти».

В основной области – заголовок «Соревнования» и две кнопки справа: зелёная «Результаты» и синяя «Создать соревнование».

Ниже – белый блок фильтра по статусу с вкладками: активная «Все» – синяя с белым текстом, остальные – серые («Предстоящие», «Идут сейчас», «Завершённые», «Отменённые»). Рядом – переключатель «Список» / «Карточки».

Под ними – белая панель поиска и фильтров: поле поиска по названию, выбор вида спорта, даты «с» и «по», кнопка «Сбросить» с серой обводкой.

Основное содержимое – Таблица на белом фоне: колонки «Название», «Спорт», «Даты», «Локация», «Статус». строки с названием соревнования, голубой меткой «Предстоящее». В режиме «Карточки» – сетка карточек с обложкой и кнопкой «Подробнее». Внизу – пагинация.

Рисунок 14 – Макет страницы тренировок преподавателя

На рисунке 15 представлен макет страницы «Студента» для преподавателя. Фон – светло-серый. Слева – белая боковая панель, активен пункт «Студента», внизу – красная кнопка «Выйти».

В основной области – заголовок «Студенты» и блок фильтров в одной строке: поле поиска по фамилии, выпадающий список «Группа», переключатели «Статус» – «Все», «Физические организаторы», «Не физические организаторы».

Ниже – белый блок группы студентов с голубой шапкой: название «Группа», счетчик, кнопка «Расписание группы». Под шапкой Таблица: Фамилия, Имя, Отчество, Логин, Статус физического организатора, в колонке действий ссылки «Профиль студента», «Сделать физическим организатором» и «Убрать физического организатора», Внизу – пагинация.

Панель преподавателя

- Главная
- Профиль
- Соревнования
- Результаты
- Лента фотографий
- Тренировки
- Новости
- Команды
- Студенты
- Виды спорта

Студенты

Фамилия
Группа:

Все группы ▼

Статус:

Все

Физорги

Не физорги

Группа

12

Расписание группы

Фамилия	Имя	Отчество	Логин	Статус	Действия
Иванов	Иван		ivanov	Физорг	Профиль студента Убрать физорга
Петров				Студент	Сделать физоргом
Сидоров					
...					

1

2

3

Выйти

Рисунок 15 – Макет страницы «Студенты»

На рисунке 16 представлена страница «Виды спорта» в макете для преподавателя. Фон – светло-серый. Слева – боковая панель, активен пункт «Виды спорта» (синий), внизу – кнопка «Выйти».

В основной области – заголовок «Виды спорта» и синяя кнопка «Создать вид спорта» справа. Ниже – белая панель поиска (поле «Введите название», кнопки «Найти» и «Сбросить») и переключатель «Список» / «Карточки».

Основное содержимое в режиме «Список» – белая Таблица: колонки «Название», «Описание», «Создатель», «Действия»; в действиях ссылки «Подробнее» (синяя), «Редактировать» (фиолетовая), «Удалить» (красная).

В режиме «Карточки» – сетка карточек с названием, описанием, создателем и кнопками «Подробнее» (синяя), «Редактировать» (серая обводка), «Удалить» (красная). Внизу – пагинация.

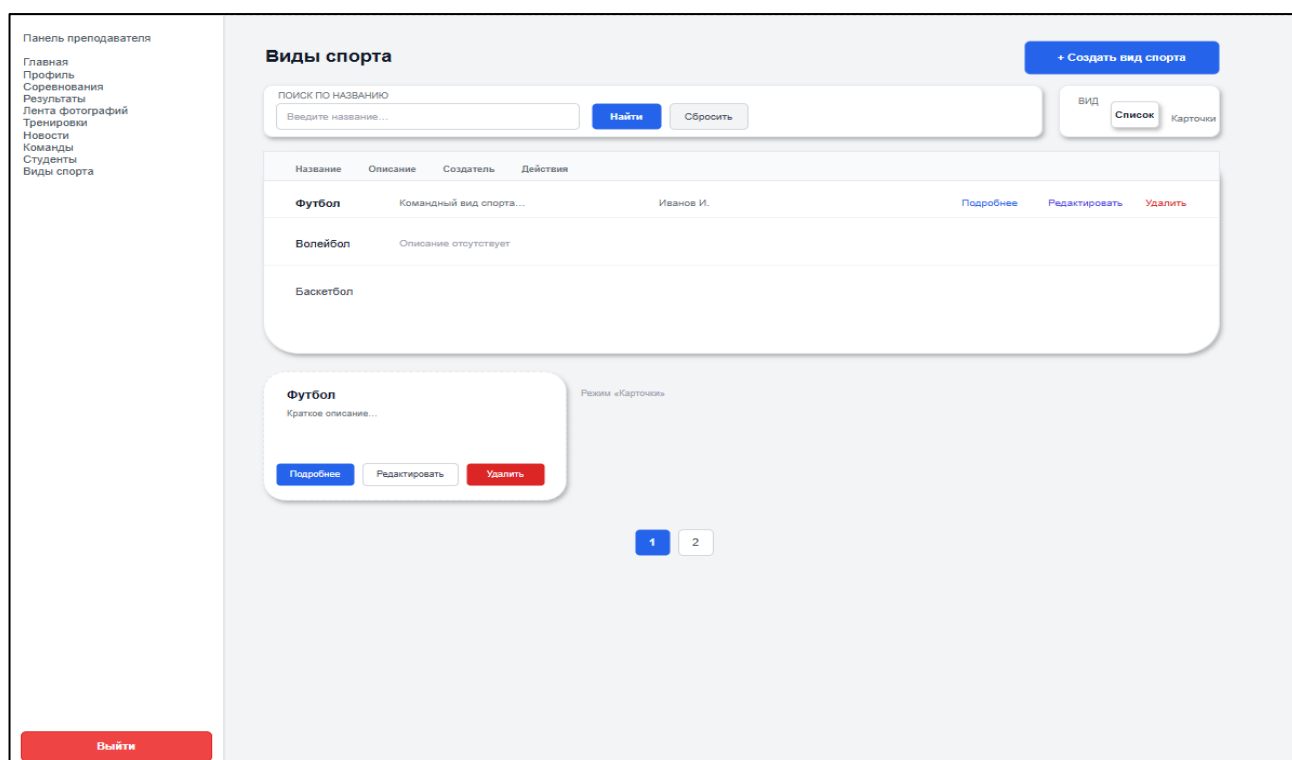


Рисунок 16 – Макет страницы видов спорта преподавателя

Спроектированный интерфейс задаёт основу для реализации: определены состав экранов, повторяющиеся компоненты, логика навигации и визуальный стиль, что позволяет на следующем этапе разработки последовательно переносить решения в программный продукт с сохранением единообразия и удобства для всех категорий пользователей.

4 Реализация программного продукта

4.1 Реализация базы данных

Реализация базы данных веб-приложения спортивного клуба «Авиатор» выполнялась на основе ER-модели, разработанной на этапе проектирования. В качестве системы управления базами данных используется MySQL, а управление структурой базы данных осуществляется средствами миграций фреймворка Laravel.

Для подключения веб-приложения к базе данных используются параметры, указанные в конфигурационном файле .env. В данном файле задаются имя базы данных, адрес сервера, порт подключения, имя пользователя и пароль. Пример настройки подключения к базе данных представлен на рисунке 17.

```
DB_CONNECTION=mysql
DB_HOST=web.edu
DB_PORT=3306
DB_DATABASE=22101_Aviators
DB_USERNAME=*****
DB_PASSWORD=*****
```

Рисунок 17 – Подключение к базе данных

Миграции Laravel представляют собой PHP-файлы, описывающие изменения структуры базы данных. Они позволяют создавать, изменять и удалять таблицы и столбцы без ручного написания SQL-запросов, а также хранить историю изменений структуры базы данных в системе контроля версий.

В рамках реализации базы данных были созданы таблицы для хранения:

- пользователей и их ролей;
- видов спорта;
- спортивных команд и участников команд;

- тренировочных сессий и записей на тренировки;
- соревнований, заявок на участие и результатов;
- новостей и изображений;
- локаций и категорий соревнований.

Связи между таблицами реализованы с помощью внешних ключей. таблица teams связана с таблицей sports, таблица team_member связывает пользователей и команды, таблица training_registrations связывает студентов с тренировочными сессиями, а таблица application_competition используется для хранения заявок пользователей на участие в соревнованиях.

На рисунке 18 представлен пример миграции таблицы teams. В данной миграции задаются поля таблицы, типы данных, первичный ключ, внешние ключи и временные метки created_at и updated_at. Запуск миграций выполняется командой php artisan migrate.

```

Aviatorss > database > migrations > 2025_11_03_043704_create_teams_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      public function up(): void
10     {
11         Schema::create('teams', function (Blueprint $table) {
12             $table->id();
13             $table->string('name', 100);
14             $table->text('description')->nullable();
15             $table->foreignId('sport_id')->nullable()->constrained('sports')->nullOnDelete();
16             $table->timestamps();
17         });
18     }
19
20     public function down(): void
21     {
22         Schema::dropIfExists('teams');
23     }
24 };

```

Рисунок 18 – Миграция таблицы «teams»

Использование миграций позволило обеспечить воспроизводимое создание структуры базы данных при развёртывании программного продукта. Таким образом, в ходе реализации базы данных была создана структура, необходимая для

хранения основных сущностей предметной области и обеспечения работы серверной части веб-приложения.

4.2 Реализация серверной части

Серверная часть веб-приложения спортивного клуба «Авиатор» реализована на языке PHP с использованием фреймворка Laravel. Серверная часть отвечает за обработку HTTP-запросов, взаимодействие с базой данных, проверку входных данных, разграничение доступа пользователей, выполнение бизнес-логики и передачу данных в представления.

Разработка серверной части выполнялась в соответствии с архитектурным шаблоном MVC (Model-View-Controller), поддерживаемым фреймворком Laravel. В данной архитектуре модель отвечает за работу с данными и связями между Таблицами, контроллер обрабатывает пользовательские запросы и выполняет бизнес-логику, а представление отвечает за отображение данных пользователю.

В серверной части реализованы следующие основные модули:

- модуль авторизации пользователей;
- модуль управления новостями;
- модуль управления видами спорта;
- модуль управления командами;
- модуль управления тренировочными сессиями;
- модуль управления соревнованиями;
- модуль обработки заявок студентов;
- модуль управления профилями пользователей;
- модуль проверки ролей и прав доступа.

Для взаимодействия с таблицами базы данных используются модели Laravel Eloquent. Модели описывают структуру данных, доступные поля и связи с другими сущностями. На рисунке 19 представлена модель «Team», используемая для работы с таблицей «teams».

```

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  class Team extends Model
9  {
10     use HasFactory;
11
12     protected $fillable = [
13         'name',
14         'description',
15         'sport_id',
16     ];
17
18     // Relationships
19     public function sport()
20     {
21         return $this->belongsTo(Sport::class);
22     }
23
24     public function members()
25     {
26         return $this->hasMany(TeamMember::class);
27     }
28
29     /** Только текущие участники (не удалённые). */
30     public function currentMembers()
31     {
32         return $this->hasMany(TeamMember::class)->whereNull('out');
33     }
34
35     public function joinRequests()
36     {
37         return $this->hasMany(TeamJoinRequest::class);
38     }
39
40     public function competitionResults()
41     {
42         return $this->hasMany(CompetitionResult::class, 'teams_id');
43     }
44 }

```

Рисунок 19 – Модель таблицы «Teams»

Модель «Team» используется в контроллерах при выполнении операций с командами: создании, просмотре, редактировании, удалении и получении связанных данных. Пример использования модели в контроллере представлен на рисунке 20.

```

namespace App\Http\Controllers;

use App\Concerns\InteractsWithLdapStudentDirectory;
use App\Models\Sport;
use App\Models\Team;
use App\Models\TeamMember;
use App\Models\User;
use Illuminate\Http\Request;
use LdapRecord\Models\ActiveDirectory\User as LdapUser;

```

Рисунок 20 – Использование модели в контроллере

Основная бизнес-логика серверной части размещается в контроллерах. Контроллеры принимают HTTP-запросы, выполняют проверку входных данных, обращаются к моделям, сохраняют или изменяют данные в базе данных и возвращают пользователю соответствующее представление или перенаправление.

На рисунке 21 представлен контроллер «TeamController», который отвечает за работу с разделом «Команды». В данном контроллере реализованы функции просмотра списка команд, создания новой команды, редактирования сведений о команде и удаления записи.

```

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Concerns\InteractsWithLdapStudentDirectory;
6  use App\Models\Sport;
7  use App\Models\Team;
8  use App\Models\TeamMember;
9  use App\Models\User;
10 use Illuminate\Http\Request;
11 use LdapRecord\Models\ActiveDirectory\User as LdapUser;
12
13 class TeamController extends Controller
14 {
15     use InteractsWithLdapStudentDirectory;
16
17     /**
18      * Display a listing of the teams.
19      */
20     public function index()
21     {
22         $teams = Team::with(['members.user', 'sport'])->latest()->paginate(10);
23
24         return view('teams.teacher.index', compact('teams'));
25     }
26
27     /**
28      * Show the form for creating a new team.
29      */
30     public function create()
31     {
32         $sports = Sport::query()->orderBy('name')->orderBy('id')->get();
33
34         return view('teams.teacher.create', compact('sports'));
35     }
36
37     /**
38      * Store a newly created team in storage.
39      */
40     public function store(Request $request)
41     {
42         $validated = $request->validate([
43             'name' => 'required|string|max:100|unique:teams,name',
44             'description' => 'nullable|string',
45             'sport_id' => 'nullable|exists:sports,id',
46         ], [
47             'name.unique' => 'Команда с таким названием уже существует.',
48         ]);

```

Рисунок 21 – Контроллер раздела «Команды»

Для проверки входных данных используются средства валидации Laravel. Валидация применяется при создании и редактировании команд, новостей, тренировочных сессий, соревнований, заявок и других сущностей. Это позволяет

исключить сохранение некорректных данных и обеспечить целостность информации в базе данных.

Маршруты веб-приложения определяют соответствие URL-адресов методам контроллеров. На рисунке 22 представлена часть маршрутов веб-приложения спортивного клуба «Авиатор». Полный код представлен в приложении Б.

```
Route::middleware(['auth', EnsureAllowedRoles::class . ':teacher'])->group(function () {
    Route::get('/teams/create', [TeamController::class, 'create'])->name('teams.create');
    Route::post('/teams', [TeamController::class, 'store'])->name('teams.store');
    Route::get('/teams/{team}/edit', [TeamController::class, 'edit'])->name('teams.edit');
    Route::put('/teams/{team}', [TeamController::class, 'update'])->name('teams.update');
    Route::patch('/teams/{team}', [TeamController::class, 'update']);
    Route::delete('/teams/{team}', [TeamController::class, 'destroy'])->name('teams.destroy');
    Route::post('/teams/{team}/members', [TeamController::class, 'addMember'])->name('teams.members.add');
    Route::get('/teams/{team}/search-students', [TeamController::class, 'searchStudents'])->name('teams.search-students');
    Route::put('/teams/{team}/members/{member}', [TeamController::class, 'updateMemberRole'])->name('teams.members.update-role');
    Route::post('/teams/{team}/members/{member}/accept', [TeamController::class, 'acceptMember'])->name('teams.members.accept');
    Route::delete('/teams/{team}/members/{member}', [TeamController::class, 'removeMember'])->name('teams.members.remove');
```

Рисунок 22 – Часть маршрутов веб-приложения

Для работы маршрутов используются подключённые контроллеры. Подключение контроллеров к маршрутам представлено на рисунке 23.

```
use App\Http\Controllers\AuthController;
use App\Http\Controllers\CompetitionController;
use App\Http\Controllers\GuestController;
use App\Http\Controllers\HomeCarouselPhotoController;
use App\Http\Controllers\LocationController;
use App\Http\Controllers\LocationTrainingController;
use App\Http\Controllers\NewsController;
use App\Http\Controllers\SportController;
use App\Http\Controllers\StudentController;
use App\Http\Controllers\TeamController;
use App\Http\Controllers\TeamJoinRequestController;
use App\Http\Controllers\TrainingSessionController;
use App\Http\Controllers\TrainingRegistrationController;
use App\Http\Middleware\EnsureAllowedRoles;
use App\Support\ParticipantListingDateFilter;
use App\Models\Competition;
use App\Models\Sport;
use App\Models\Team;
use App\Models\TrainingSession;
use Illuminate\Http\Request;
use Illuminate\Pagination\LengthAwarePaginator;
use Illuminate\Support\Facades\Route;
use Illuminate\Validation\Rule;
```

Рисунок 23 – Подключение контроллеров к маршрутам

Разграничение доступа реализовано с использованием middleware и проверки роли авторизованного пользователя. Для пользователей с ролью «Студент» и «Преподаватель» предусмотрены разные наборы доступных маршрутов и операций. При попытке обращения к запрещённому разделу система ограничивает доступ и отображает сообщение о невозможности выполнения действия.

Таким образом, в ходе реализации серверной части были разработаны модели, контроллеры, маршруты и механизмы проверки доступа, обеспечивающие обработку пользовательских запросов, взаимодействие с базой данных и выполнение основной бизнес-логики веб-приложения.

4.3 Реализация клиентской части

Клиентская часть веб-приложения спортивного клуба «Авиатор» реализована как многостраничное приложение (MPA) на базе Laravel и шаблонизатора Blade. При переходе между разделами браузер загружает новую HTML-страницу с сервера. Для авторизованных пользователей повторно используется общая оболочка интерфейса, включающая боковое меню, область отображения данных и элементы управления пользователем.

В качестве основы пользовательского интерфейса используются Blade-шаблоны. Для разных групп пользователей реализованы отдельные базовые шаблоны:

- `guest.blade.php` – шаблон публичной части для неавторизованных пользователей;
- `student.blade.php` – шаблон панели студента;
- `teacher.blade.php` – шаблон панели преподавателя.

Такое разделение позволяет отображать разные наборы пунктов меню, элементов управления и доступных действий в зависимости от роли пользователя. На рисунке 24 представлен шаблон `guest.blade.php`, используемый для отображения публичной части веб-приложения.

```

1 <!DOCTYPE html>
2 <html lang="ru" class="overflow-x-hidden">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <meta http-equiv="X-UA-Compatible" content="IE=edge">
7
8   <title>@yield('title', 'Главная') - Авиатор</title>
9
10  <meta name="description" content="@yield('description', 'Спортивный клуб Авиатор')">
11
12  <!-- Styles -->
13  <script src="https://cdn.tailwindcss.com"></script>
14
15  @stack('styles')
16  @yield('head')
17 </head>
18 <body class="m-0 bg-gray-100 flex flex-col min-h-screen overflow-x-hidden">
19   <!-- Навигация -->
20   <nav class="bg-white shadow-md">
21     <div class="mx-auto max-w-[min(120rem,calc(100vw-2rem))] px-4 sm:px-6 lg:px-8">
22       <div class="flex justify-between items-center h-16">
23         <div class="flex items-center">
24           <a href="{{ route('home') }}" class="text-xl font-bold text-gray-800 hover:text-blue-600 transition">
25             Авиатор
26           </a>
27         </div>
28
29         <div class="hidden md:flex items-center space-x-4">
30           <a href="{{ route('home') }}" class="text-gray-700 hover:text-blue-600 px-3 py-2 rounded-md text-sm font-medium transition">
31             Главная
32           </a>
33           <a href="{{ route('guest.news') }}" class="text-gray-700 hover:text-blue-600 px-3 py-2 rounded-md text-sm font-medium transition">
34             Новости
35           </a>
36           <a href="{{ route('guest.sports') }}" class="text-gray-700 hover:text-blue-600 px-3 py-2 rounded-md text-sm font-medium transition">
37             Виды спорта
38           </a>
39           <a href="{{ route('guest.teams') }}" class="text-gray-700 hover:text-blue-600 px-3 py-2 rounded-md text-sm font-medium transition">
40             Команды
41           </a>
42           <a href="{{ route('guest.results') }}" class="text-gray-700 hover:text-blue-600 px-3 py-2 rounded-md text-sm font-medium transition">
43             Результаты соревнований
44           </a>
45           <a href="{{ route('login') }}" class="bg-blue-500 text-white px-4 py-2 rounded-lg hover:bg-blue-600 transition duration-200 text-sm font-medium">
46             Войти
47           </a>
48         </div>

```

Рисунок 24 – Базовая оболочка интерфейса панели гостя

Навигация в веб-приложении реализована с использованием бокового меню. Состав пунктов меню зависит от роли авторизованного пользователя. Для студента отображаются разделы, связанные с просмотром спортивной информации и подачей заявок. Для преподавателя отображаются дополнительные разделы управления данными.

На страницах со списками реализованы элементы поиска, фильтрации и постраничного вывода данных. Такие элементы используются на страницах соревнований, тренировочных сессий, студентов, видов спорта и команд. Это позволяет пользователю быстрее находить необходимые записи и работать с большим объёмом данных.

Формы клиентской части обеспечивают ввод данных пользователем и отображение сообщений об ошибках валидации, полученных от серверной части.

При успешном выполнении операции пользователь получает уведомление или перенаправляется к соответствующему разделу веб-приложения.

Входная точка клиентской части задаётся файлом `resources/js/app.js`, который подключает модуль `resources/js/bootstrap.js`. Данный модуль используется для подключения общих JavaScript-настроек приложения и подготовки клиентских скриптов к работе.

Таким образом, клиентская часть веб-приложения реализует пользовательский интерфейс для неавторизованных пользователей, студентов и преподавателей, обеспечивает навигацию по разделам, работу с формами, отображение списков данных и взаимодействие пользователя с серверной частью приложения.

5 Тестирование программного продукта

5.1 Функциональное тестирование

Тестирование – это процесс проверки программного продукта на основе набора тестов путём сопоставления фактических результатов выполнения программы с ожидаемыми результатами. Целью функционального тестирования является проверка корректности работы веб-приложения спортивного клуба «Авиатор» в соответствии с функциональными требованиями.

В рамках функционального тестирования были проверены основные пользовательские сценарии, обработка некорректных данных и разграничение доступа по ролям. Функциональные тесты были реализованы с использованием PHPUnit и средств тестирования Laravel.

В Таблицах 26 – 31 отражены сценарии функционального тестирования.

Таблица 26 – Тестовый сценарий FUNC-POS-01

Параметр	Описание
ID теста	FUNC-POS-01.
Название теста	Студенты: преподаватель открывает список студентов
Предусловия	Студенты: преподаватель открывает список студентов
Шаги	1) Авторизоваться под преподавателем. 2) Открыть раздел «Студенты» (GET /students). 3) Проверить содержимое страницы.
Тестовые данные	Преподаватель (роль teacher, active = true); студент с group_name = «ИС-221», status_fizorg = false.
Ожидаемый результат	HTTP 200; заголовок «Студенты»; в таблице отображается фамилия студента из тестовых данных.

Таблица 27 – Тестовый сценарий FUNC-POS-02

Параметр	Описание
ID теста	FUNC-POS-02.
Название теста	Студенты: преподаватель открывает профиль студента
Предусловия	Пользователь с ролью teacher авторизован. В учете существует студент с известным id.
Шаги	1) Авторизоваться под преподавателем 2) Открыть профиль студента 3) Проверить отображаемые ФИО
Тестовые данные	Преподаватель, студент с firstname и lastname, group_name.

Продолжение таблицы 27

Ожидаемый результат	HTTP 200, на странице профиля отображаются имя и фамилия выбранного студента
---------------------	--

Таблица 28 – Тестовый сценарий FUNC-POS-03

Параметр	Описание
ID теста	FUNC-POS-03.
Название теста	Студенты: преподаватель назначает студента физоргом
Предусловия	Пользователь с ролью teacher авторизован; студент существует, статус физического организатора должен быть false, в его группе нет другого физорга.
Шаги	1) Убедиться, что у студента «Не физический организатор» 2) Авторизоваться под преподавателем. 3) Выполнить действие «Сделать физоргом» 4) Дождаться ответа.
Тестовые данные	Преподаватель; студент с группы «ИС-22-1» .
Ожидаемый результат	«Статус физорга успешно установлен!»

Таблица 29 – Тестовый сценарий FUNC-NEG-VAL-01

Параметр	Описание
ID теста	FUNC-NEG-VAL-01
Название теста	Студенты: отклонение недопустимого значения фильтра «физорг».
Предусловия	Пользователь с ролью teacher авторизован; раздел «Студенты» доступен.
Шаги	1) Авторизоваться под преподавателем. 2) Открыть список студентов с параметром fizorg=not_a_valid_filter (GET /students?fizorg=not_a_valid_filter). 3) Проверить результат
Тестовые данные	Преподаватель. Недопустимое значение фильтра fizorg = «not_a_valid_filter»
Ожидаемый результат	Отклоненный запрос валидацией. В сессии ошибка по полю fizorg.

Таблица 30 – Тестовый сценарий FUNC-NEG-ROLE-01

Параметр	Описание
ID теста	FUNC-NEG-ROLE-01
Название теста	Студенты: студент не может открыть список студентов (роль)
Предусловия	Пользователь с ролью student авторизован и активен; маршрут /students защищён middleware только для teacher.

Продолжение таблицы 30

Шаги	1) Авторизоваться под студентом 2) Попытаться открыть раздел «Студенты» (GET /students) 3) Проверить результат
Тестовые данные	Студент. Активность пользователя. Группа пользователя.
Ожидаемый результат	«Вход запрещен»

Таблица 31 – Тестовый сценарий FUNC-NEG-ROLE-02

Параметр	Описание
ID теста	FUNC-NEG-ROLE-02
Название теста	Студенты: студент не может изменить статус физорга (роль)
Предусловия	Пользователь с ролью student авторизован; целевой студент существует, status_fizorg = false.
Шаги	1) Авторизоваться под студентом 2) Отправить запрос назначения физорга (POST /students/{id}/toggle-fizorg). 3) Проверить результат
Тестовые данные	Студент-инициатор и целевой студент.
Ожидаемый результат	«HTTP 403»

На рисунке 25 изображен код тест-кейса FUNC-POS-01.

```
public function func_pos_01(): void
{
    $response = $this->actingAs($this->teacher)->get(route('students.index'));

    $response->assertOk();
    $response->assertSee('Студенты', false);
    $response->assertSee($this->student->lastname, false);
}
```

Рисунок 25 – Код тест-кейса FUNC-POS-01.

На рисунке 26 изображен код тест-кейса FUNC-POS-02.

```
public function func_pos_02(): void
{
    $response = $this->actingAs($this->teacher)->get(route('students.show', $this->student));

    $response->assertOk();
    $response->assertSee($this->student->lastname, false);
    $response->assertSee($this->student->firstname, false);
}
```

Рисунок 26 – Код тест-кейса FUNC-POS-02

На рисунке 27 изображен код тест-кейса FUNC-POS-03.

```

public function func_pos_03(): void
{
    $this->assertFalse($this->student->fresh()->status_fizorg);

    $response = $this->actingAs($this->teacher)->post(
        route('students.toggle-fizorg', $this->student)
    );

    $response->assertRedirect();
    $response->assertSessionHas('success', 'Статус физорга успешно установлен!');

    $this->assertTrue($this->student->fresh()->status_fizorg);
}

```

Рисунок 27 – Код тест-кейса FUNC-POS-03

На рисунке 28 изображен код тест-кейса FUNC-NEG-01.

```

public function func_neg_01(): void
{
    $response = $this->actingAs($this->teacher)->get(route('students.index', [
        'fizorg' => 'not_a_valid_filter',
    ]));

    $response->assertSessionHasErrors(['fizorg']);
}

```

Рисунок 28 – Код тест-кейса FUNC-NEG-VAL-01.

На рисунке 29 изображен код тест-кейса FUNC-NEG-02.

```

public function func_neg_02(): void
{
    $response = $this->actingAs($this->student)->get(route('students.index'));

    $response->assertStatus(403);
}

```

Рисунок 29 – Код тест-кейса FUNC-NEG-ROLE-01

На рисунке 30 изображен код тест-кейса FUNC-NEG-03.

```

public function func_neg_03(): void
{
    $response = $this->actingAs($this->student)->post(
        route('students.toggle-fizorg', $this->student)
    );

    $response->assertStatus(403);

    $this->assertFalse($this->student->fresh()->status_fizorg);
}

```

Рисунок 30 – код тест-кейса FUNC-NEG-ROLE-02

Для запуска тестов в терминале вводим команду «php artisan test». На рисунке 31 изображен результат тестов.

```

PS C:\Users\Stuart\Desktop\Новая папка\Aviatorss> php artisan test

PASS Tests\Feature\TestFuncNeg\FuncNegRole01StudentCannotAccessStudentsListTest
✓ func neg role 01 student cannot open students list

PASS Tests\Feature\TestFuncNeg\FuncNegRole02StudentCannotToggleFizorgTest
✓ func neg role 02 student cannot toggle fizorg

PASS Tests\Feature\TestFuncNeg\FuncNegVal01StudentsInvalidFizorgFilterTest
✓ func neg val 01 students rejects invalid fizorg filter

PASS Tests\Feature\TestFuncPos\FuncPos01StudentsTeacherOpensListTest
✓ func pos 01 students teacher opens list

PASS Tests\Feature\TestFuncPos\FuncPos02StudentsTeacherOpensProfileTest
✓ func pos 02 students teacher opens profile

PASS Tests\Feature\TestFuncPos\FuncPos03StudentsTeacherAssignsFizorgTest
✓ func pos 03 students teacher assigns fizorg

Tests: 6 passed (15 assertions)
Duration: 1.92s

```

Рисунок 31 – Результаты тестов

По результатам функционального тестирования проверены основные сценарии работы веб-приложения, обработка некорректных входных данных и разграничение доступа по ролям. Все выполненные тесты завершились успешно, что подтверждает корректность реализации проверяемых функций.

5.2 Нагрузочное тестирование

Нагрузочное тестирование веб-приложения спортивного клуба «Авиатор» выполнялось с использованием инструмента Grafana k6. Тестирование

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 54
Изм.	Лист	№ докум.	Подпись	Дата		

проводилось по HTTP к развёрнутому веб-приложению и было направлено на проверку производительности и стабильности работы системы при одновременных обращениях пользователей.

Цель нагрузочного тестирования – определить, выдерживает ли серверная часть веб-приложения одновременную работу пользователей без значительного увеличения времени ответа и роста количества ошибок.

Основные задачи нагрузочного тестирования:

- смоделировать нарастающую нагрузку на процесс авторизации и открытие основных страниц веб-приложения;
- зафиксировать показатели времени ответа, количества запросов и доли ошибок;
- сопоставить фактические метрики с критериями успешного прохождения теста;
- определить ограничения тестового окружения.

Таблица 32 – Сценарий нагрузочного тестирования

Поле	Описание
Название теста	Нагрузочное тестирование входа в систему (LDAP, веб-сессия Laravel)
Цель тестирования	Проверить производительность и стабильность цепочки авторизации при нарастающем числе одновременных пользователей (по умолчанию до 10 VU).
Сценарий тестирования	Исполнитель ramping-vus 1) рост 0→~3VU за 30с; 2) рост →10VU за 1мин; 3) удержание 10VU за 1мин; 4) снижение 10→0VU за 30с; плавное завершение 30с. В итерации: GET /login (CSRF) → POST /login (LDAP) → GET /dashboard → POST /logout; пауза 1–3 с (случайно).
Тестовые данные	K6_LOGIN и K6_PASSWORD в корневом .env или load/.env (учётная запись LDAP в разрешённой группе AD); BASE_URL (например http://127.0.0.1:8000).
Критерии успеха (из k6)	Доля неуспешных HTTP-запросов < 10% (http_req_failed); p(95) общего времени запроса < 5 с; p(95) для POST /login < 8 с; доля успешных проверок (checks) > 80%.

На рисунке 32 изображен результат нагрузочного тестирования.

```
■ TOTAL RESULTS

checks_total.....: 2826    15.46385/s
checks_succeeded...: 100.00% 2826 out of 2826
checks_failed.....: 0.00%   0 out of 2826

✓ login page is 200
✓ csrf token present
✓ login redirects (302)
✓ redirect to dashboard
✓ not rejected to login page
✓ dashboard after login is 200

HTTP
http_req_duration.....: avg=112.76ms min=11.66ms med=88.12ms max=503.11ms p(90)=236.29ms p(95)=299.35ms
  { expected_response:true }...: avg=112.76ms min=11.66ms med=88.12ms max=503.11ms p(90)=236.29ms p(95)=299.35ms
  { name:POST /login }.....: avg=119.48ms min=20.24ms med=92.57ms max=483.53ms p(90)=242.05ms p(95)=324.42ms
http_req_failed.....: 0.00%   0 out of 2355
http_reqs.....: 2355    12.886541/s

EXECUTION
iteration_duration.....: avg=2.51s    min=1.14s    med=2.52s    max=4.34s    p(90)=3.42s    p(95)=3.72s
iterations.....: 470    2.571836/s
```

Рисунок 32 – Результаты нагрузочного тестирования

По результатам нагрузочного тестирования было установлено, что веб-приложение сохраняет работоспособность при заданной нагрузке. Фактические значения времени ответа и доли ошибок сопоставлены с установленными критериями успешного прохождения теста.

6 Руководство пользователя программного продукта

6.1 Инструкция по установке и запуску программного продукта

На таблице 33 изображена Таблица с требованиями к программному окружению для запуска веб-приложения.

Таблица 33 – Требования к программному окружению

Компонент	Версия
PHP	8.2 +
Composer	2.X
Node.js	20.X LTS и выше
npm	10.x и выше
MySQL	8.x
LDAP	Active Directory, доступ к каталогу Auth iat.iat

Для установки зависимостей требуется прописать следующие команды:

- Composer install – установка зависимых пакетов.
- Npm install – установка зависимостей клиентской части.
- Composer run setup – быстрая первичная настройка.

Настройка окружения:

- Скопируйте при помощи команды «`сору .env.example .env`», также требуется отредактировать этот файл.

- Выполните команду «`php artisan key:generate`» для генерации ключа.

Для базы данных выполните миграции при помощи команды «`php artisan migrate`».

Соберите клиентскую часть при помощи команды «`npm run build`».

Запустите веб-сервер проекта при помощи команды «`php artisan serve`».

Запустите веб-сервер клиентской части при помощи команды «`npm run dev`».

Затем необходимо перейти по домену aviator.irkat.ru.

6.1 Руководство пользователя для роли Студент

Авторизация в веб-приложении спортивного клуба «Авиатор». На рисунке 33 изображена форма авторизации веб-приложения.

Рисунок 33 – Форма авторизации

Требуется ввести логин и пароль выданные студентам и преподавателям образовательным учреждением ГБПОУИО «ИАТ» и нажать кнопку «Войти».

В верхнем левом углу имеется кнопка меню. По нажатию открывается боковое меню. На широком экране меню отображается постоянно слева; кнопкой со стрелкой его можно свернуть. В боковом меню отображаются разделы, доступные студенту. В нижней части меню указаны имя, фамилия, роль «Студент» и кнопка «Выйти». На рисунке 34 представлена главная страница.

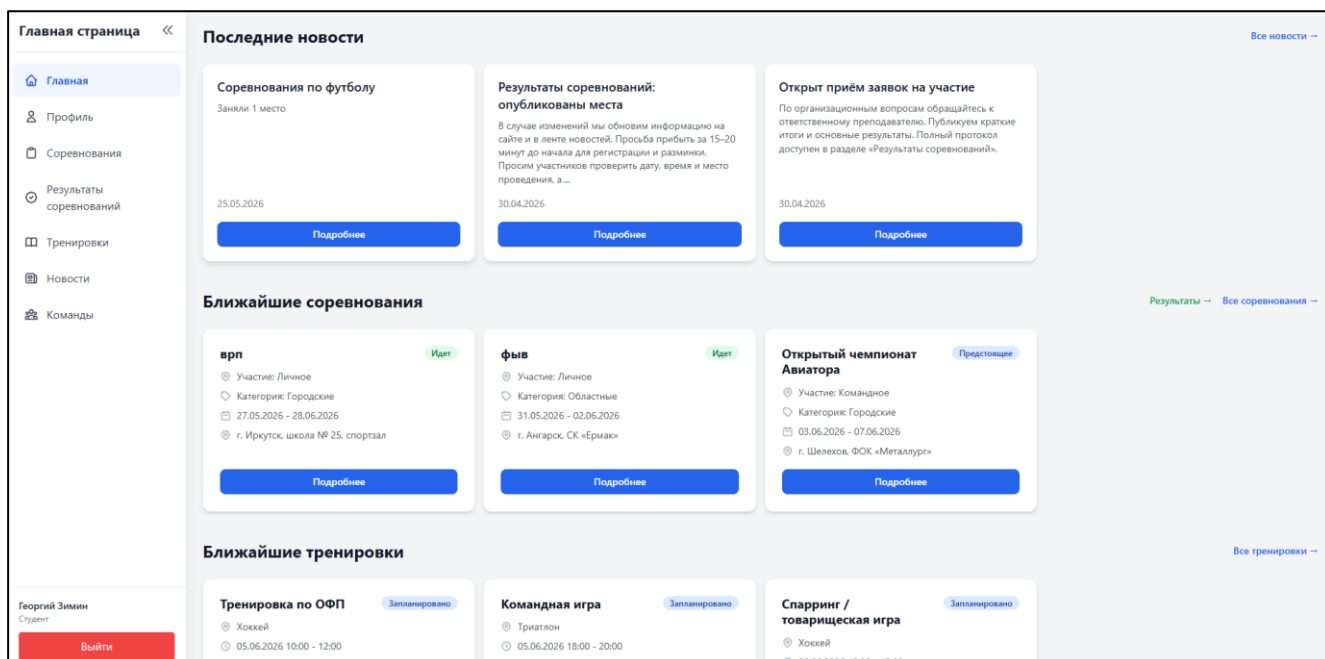


Рисунок 34 – Главная страница

По нажатию на пункт «Профиль» в боковом меню открывается личный кабинет.

На странице отображаются:

- Фотография (аватар) или заглушка; кнопка «Изменить фотографию».
- Основная информация: логин, имя, фамилия, отчество, учебная группа.
- Дополнительная информация: статус физорга группы (да/нет).
- Блок «Статистика» с количеством участия в командах, завершённых соревнованиях и тренировках.

По нажатию на карточки статистики открываются страницы истории участия:

- Команды – список команд, в которых студент состоял или состоит.
- Соревнования – завершённые соревнования с участием студента (доступны фильтры по виду спорта и датам)
- Тренировки – завершённые тренировки, на которые студент был записан.

На рисунке 35 представлена страница вкладки «Профиль».

Панель студента <<

Главная

Профиль

Соревнования

Результаты соревнований

Тренировки

Новости

Команды

Георгий Зимин

Студент

Выйти

Личный кабинет

Изменить фотографию

Основная информация

Логин

22101

Имя

Георгий

Фамилия

Зимин

Отчество

Муйдинжонович

Группа

ИС-22-2

Дополнительная информация

Статус физического организатора

Нет

Статистика

Команды

1

Соревнования

0

Тренировки

0

Внимание:

Данные в личном кабинете доступны только для просмотра. Для изменения информации обратитесь к администратору.

Рисунок 35 – Страница личного кабинета

По нажатию на «Соревнования» открывается список соревнований. Доступны фильтры (статус, вид спорта, период дат, поиск). По нажатию на соревнование открывается карточка соревнования.

На карточке отображаются описание, даты, место, состав участников (после одобрения), блок «Моя заявка».

Для предстоящего соревнования студент может нажать «Подать заявку на участие». Заявка отправляется преподавателю на рассмотрение. Отображаются статусы: ожидание, принята, отклонена (с возможностью подать заявку повторно, если отклонена).

Для соревнований со статусом «Идет» или «Завершено» подача заявки недоступна.

На рисунке 36 представлена страница соревнования с блоком заявки.

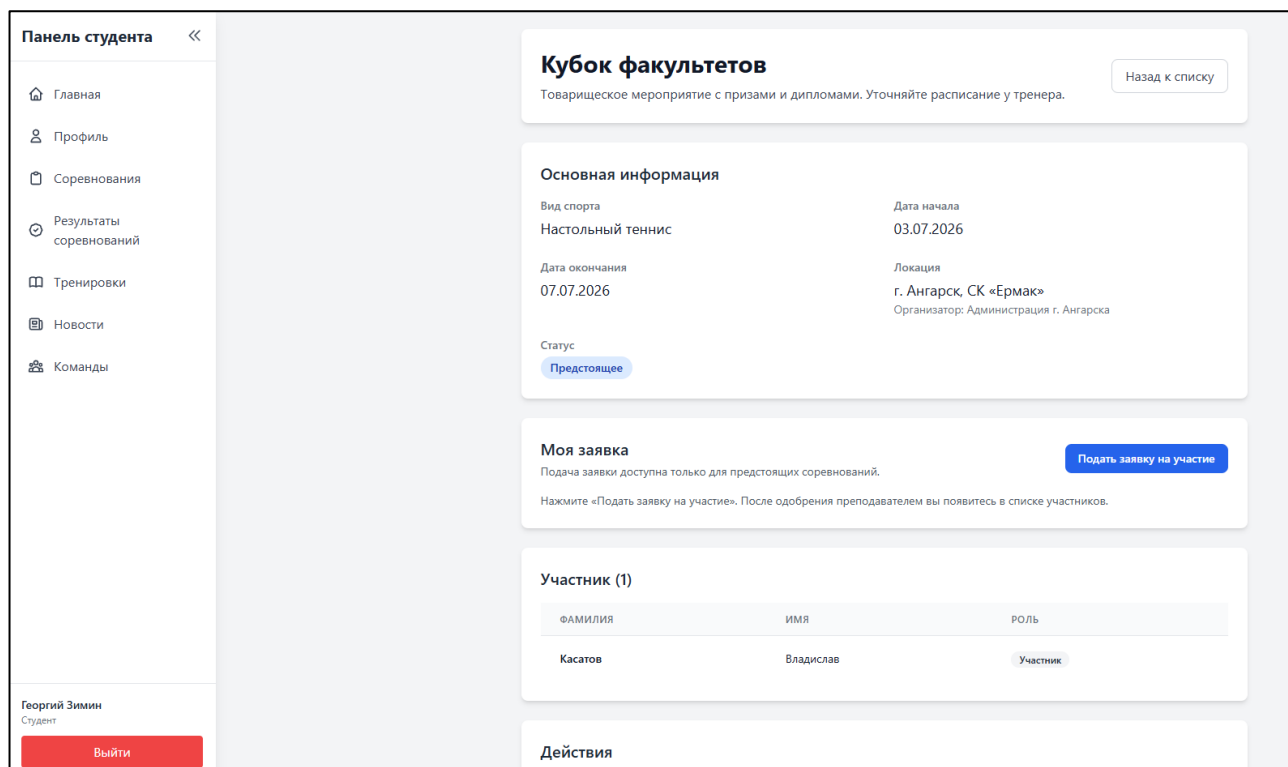


Рисунок 36 – Страница соревнования

По нажатию на «Результаты соревнований» открывается страница с итогами завершённых соревнований. Отображаются места, команды и участники. Имеются фильтры для удобного поиска нужного соревнования или вида спорта. На рисунке 37 представлена страница результатов.

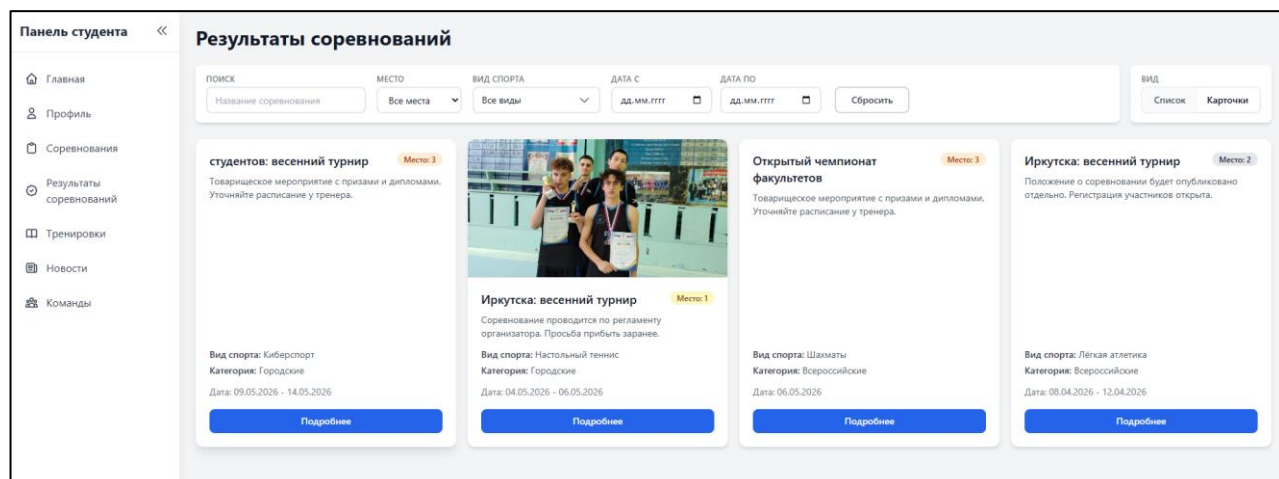


Рисунок 37 – Страница с результатом соревнований

По нажатию на «Тренировки» открывается список тренировочных сессий. Для каждой указаны название, вид спорта, время, место и статус.

На странице отдельной тренировки или в списке для доступных сессий студент может:

- нажать «Записаться» – записаться на тренировку;
- отменить запись – если запись уже выполнена, и отмена разрешена правилами системы.

На рисунке 38 представлен раздел тренировок.

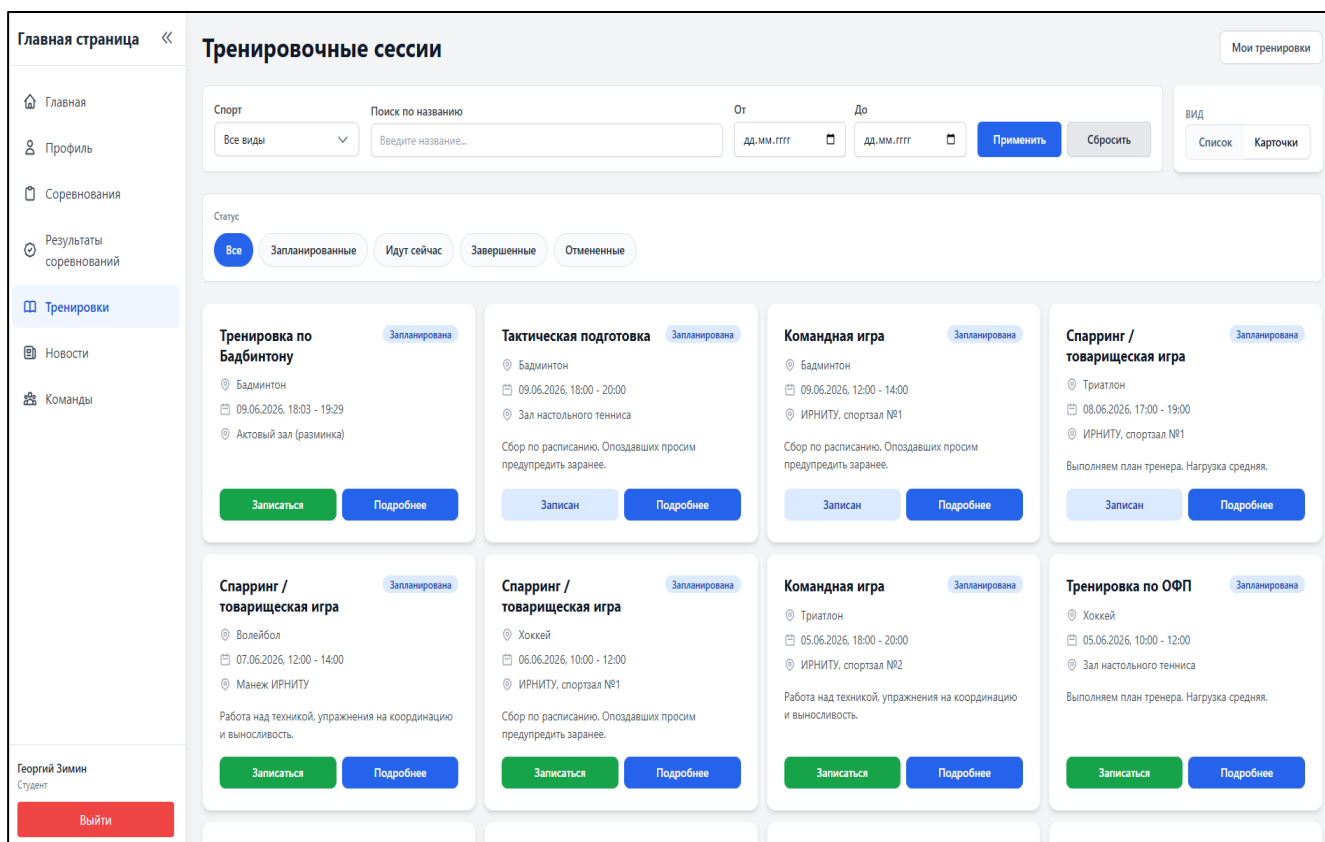


Рисунок 38 – Страница тренировок

По нажатию на «Новости» открывается список опубликованных новостей спортивного клуба. По нажатию на заголовок открывается полный текст новости с изображениями.

На рисунке 39 представлен раздел новостей.

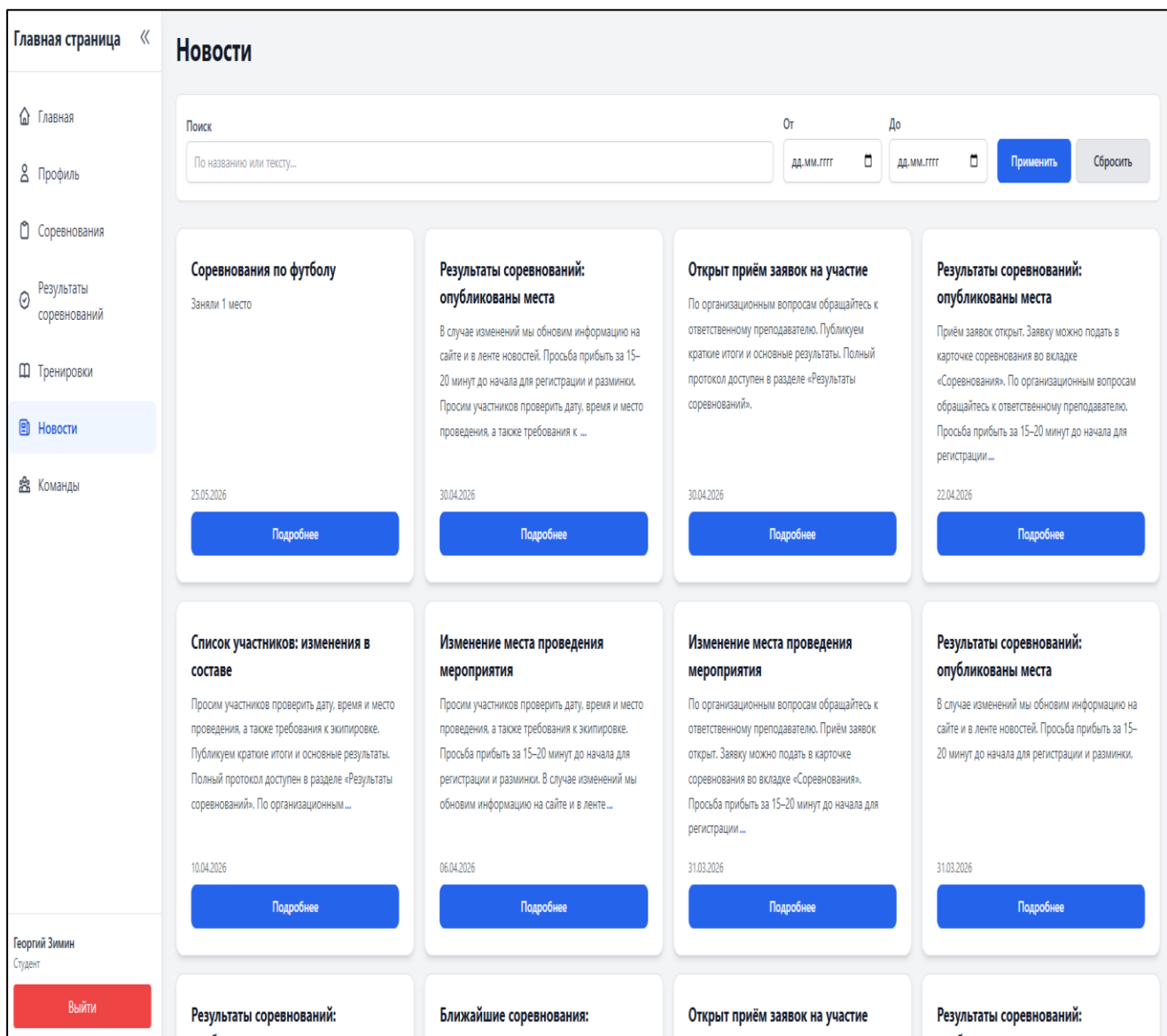


Рисунок 39 – Страница раздела Новостей

По нажатию на «Команды» отображается список спортивных команд. По нажатию на команду открывается карточка: состав, вид спорта, описание.

Если студент не состоит в команде и набор открыт, доступна кнопка «Подать заявку» на вступление. После отправки заявка ожидает решения преподавателя (одобрение или отклонение). При наличии активной заявки отображается соответствующее сообщение.

На рисунке 40 представлена карточка команды.

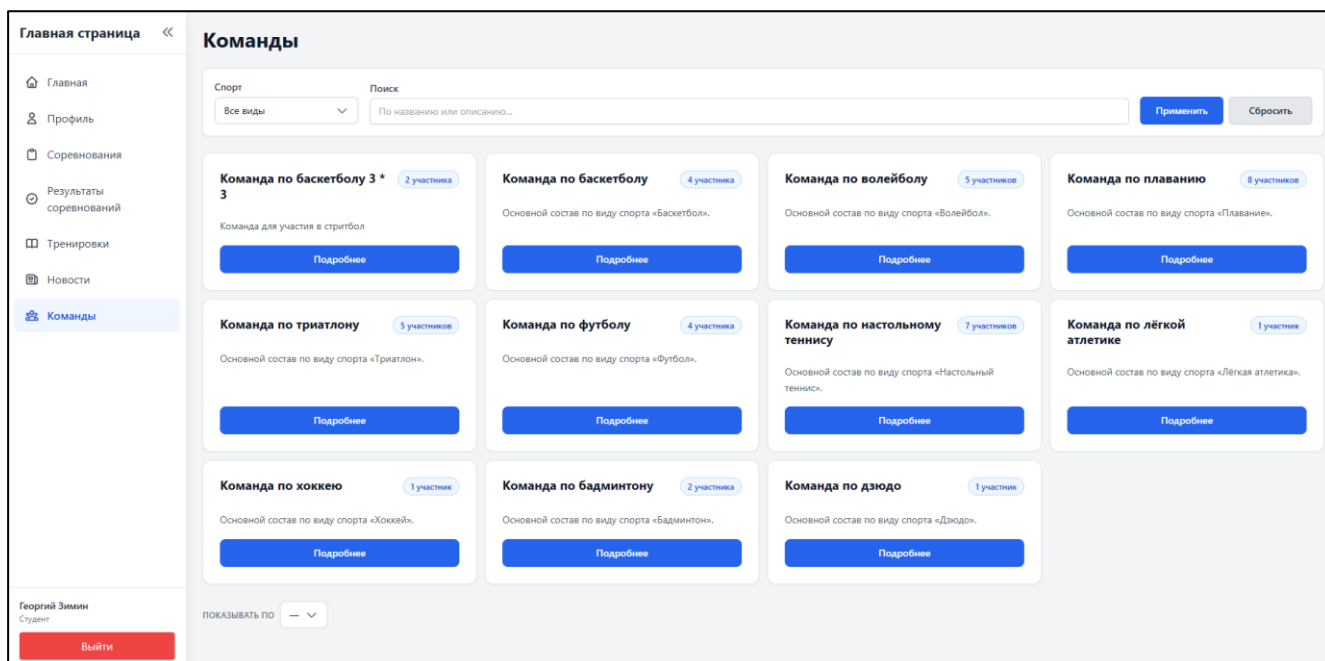


Рисунок 40 – Страница раздела Команды

6.2 Руководство пользователя для роли Преподаватель

Вход происходит также как у пользователя с ролью «Студент».

По нажатию открывается боковое меню. На широком экране меню закреплено слева.

В нижней части бокового меню отображаются имя, фамилия, роль «Преподаватель» и кнопка «Выйти». На рисунке 43 – боковое меню преподавателя.

На главной отображаются:

- Последние новости (до трёх) со ссылкой «Все новости»;
- Ближайшие соревнования – карточки с переходом в раздел соревнований и в результаты;
- Ближайшие тренировки – с переходом в раздел тренировок.

Содержание аналогично панели студента; преподаватель дополнительно из меню переходит к созданию и редактированию сущностей.

На рисунке 41 представлена главная страница.

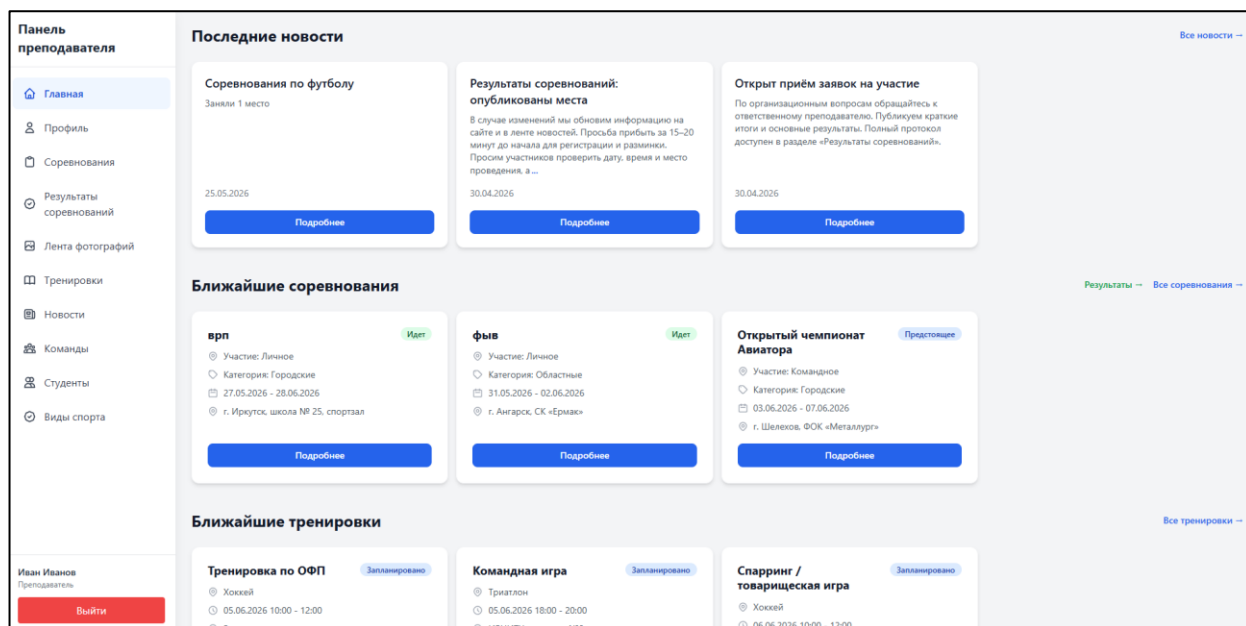


Рисунок 41 – Главная страница

По пункту «Профиль» открывается личный кабинет преподавателя: аватар (кнопка «Изменить фотографию»), логин, ФИО, служебная информация.

Редактирование персональных данных LDAP в интерфейсе, как у студента, недоступно – только просмотр и смена аватара, страница представлена на рисунке 42.

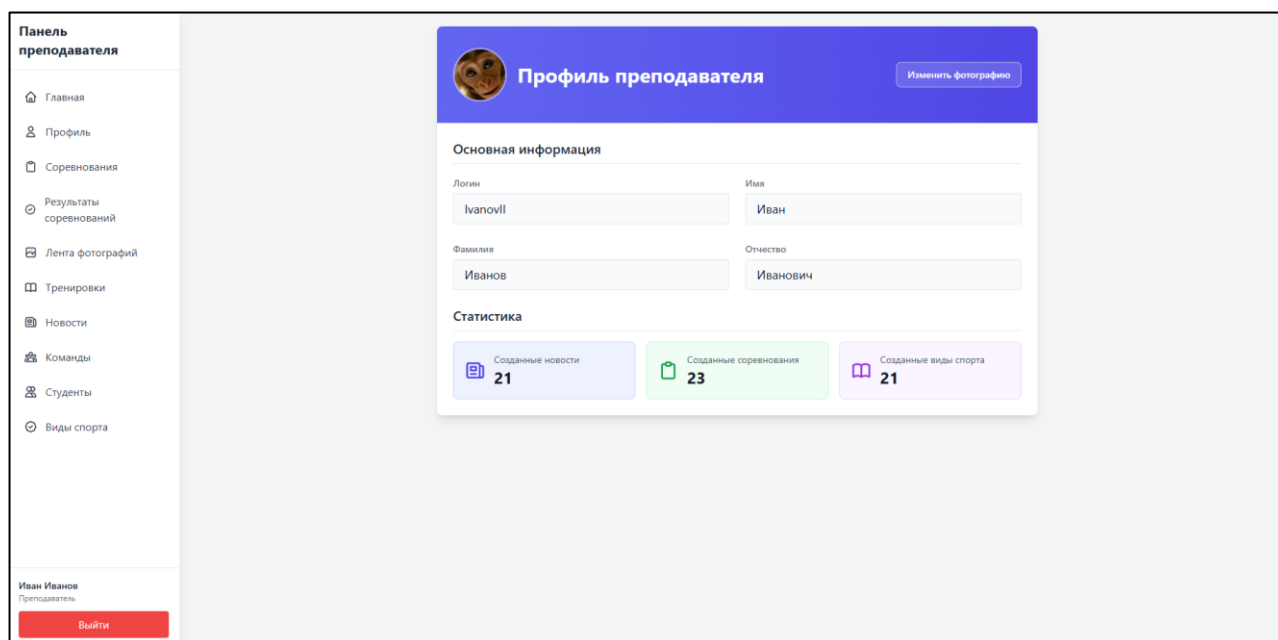


Рисунок 42 – Страница Профиля

В разделе «Соревнования» список соревнований с фильтрами; кнопки создания и перехода к редактированию.

Карточка соревнования (управление):

- изменение данных, отмена соревнования;
- заявки студентов – принять / отклонить;
- участники – добавление, смена роли, удаление;
- результаты – ввод мест и показателей;
- фотографии соревнования;
- генерация приказов (освобождение от занятий и другое – формы с датой, сопровождающим, подписантами).

Студенты подают заявки; преподаватель их обрабатывает и формирует итоговый состав. Отображено на рисунках 43-44.

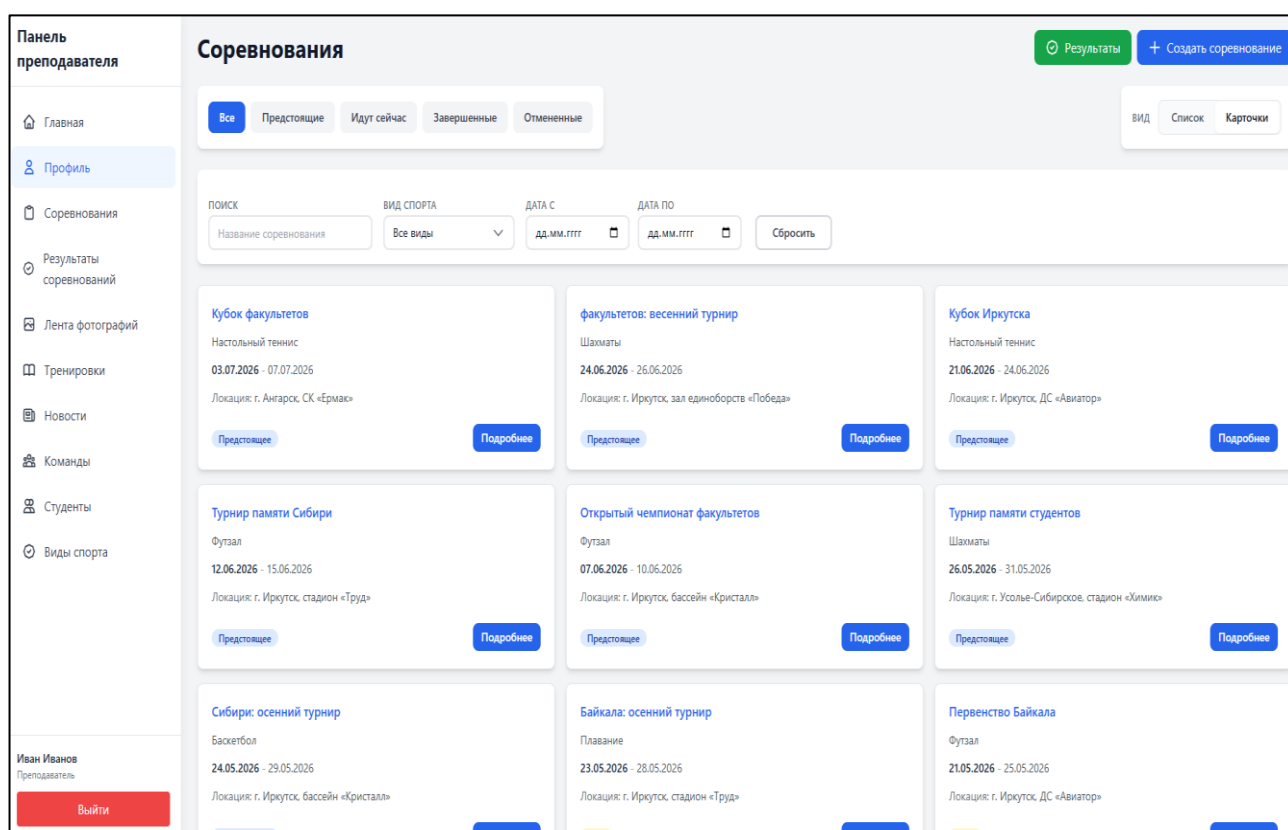


Рисунок 43 – Страница соревнований

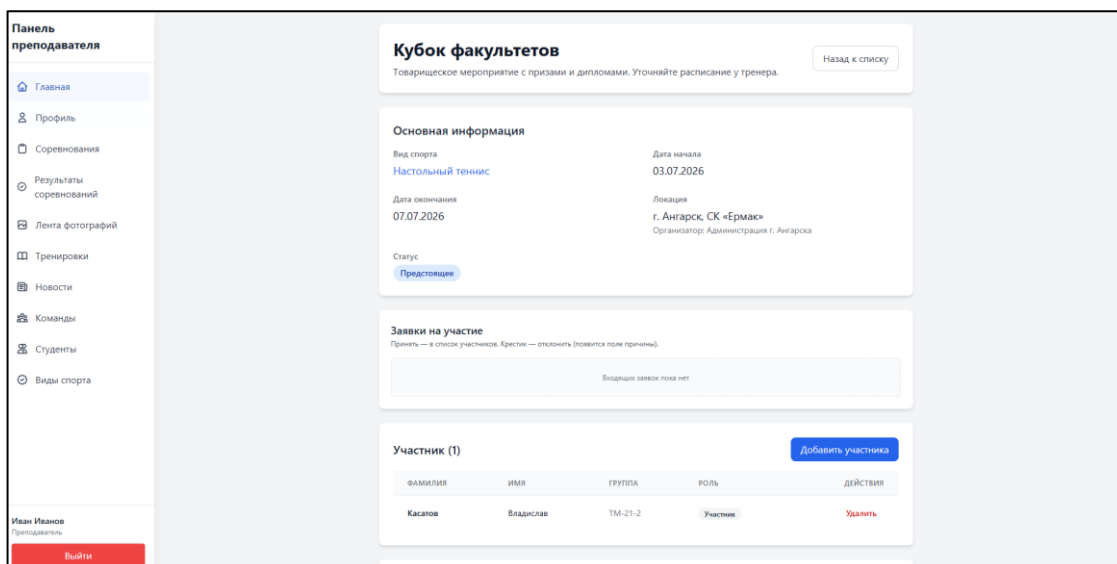


Рисунок 43 – Страница подробнее соревнования

Просмотр и при необходимости правка итогов завершённых соревнований (как у студента, с расширенными правами редактирования у ответственного преподавателя). На рисунке 49 представлена страница.

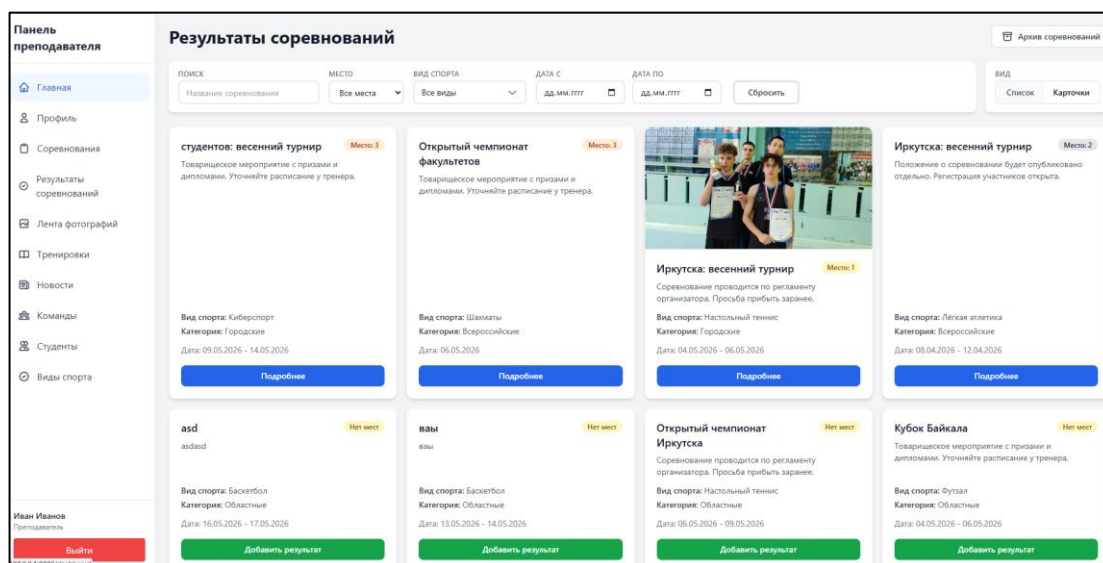


Рисунок 44 – Страница результаты соревнований

Список тренировочных сессий: создание, редактирование, отмена тренировки.

Указываются вид спорта, команда, время, площадка, статус. Студенты записываются на тренировки самостоятельно, преподаватель видит список записавшихся на странице сессии.

При необходимости – правление локациями тренировок (справочник площадок). Представлена на странице 45-46.

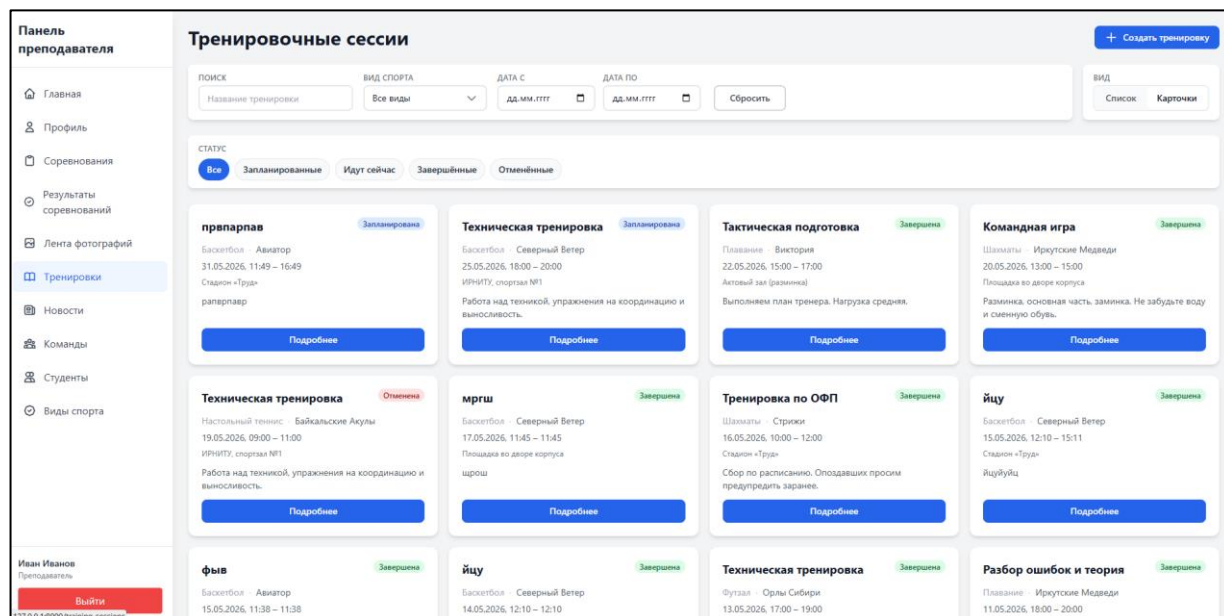


Рисунок 45 – Страница тренировочных сессий

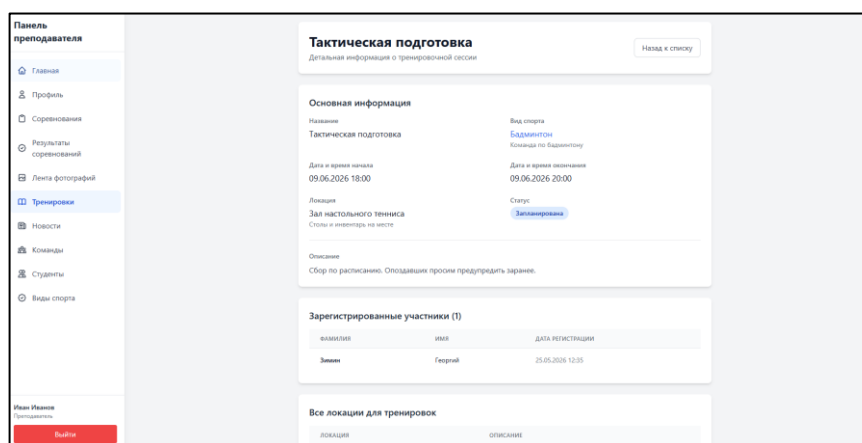


Рисунок 46 – Страница подробнее тренировочной сессии

Список всех новостей (черновики и опубликованные) находятся в разделе «Новости».

Действия преподавателя:

- Создать новость – заголовок, текст, дата, изображения.
- Редактировать черновик или опубликованную.
- Опубликовать – перевод в статус, видимый студентам и на публичной главной.
- Удалить новость.

На рисунке 47 представлен раздел «Новости».

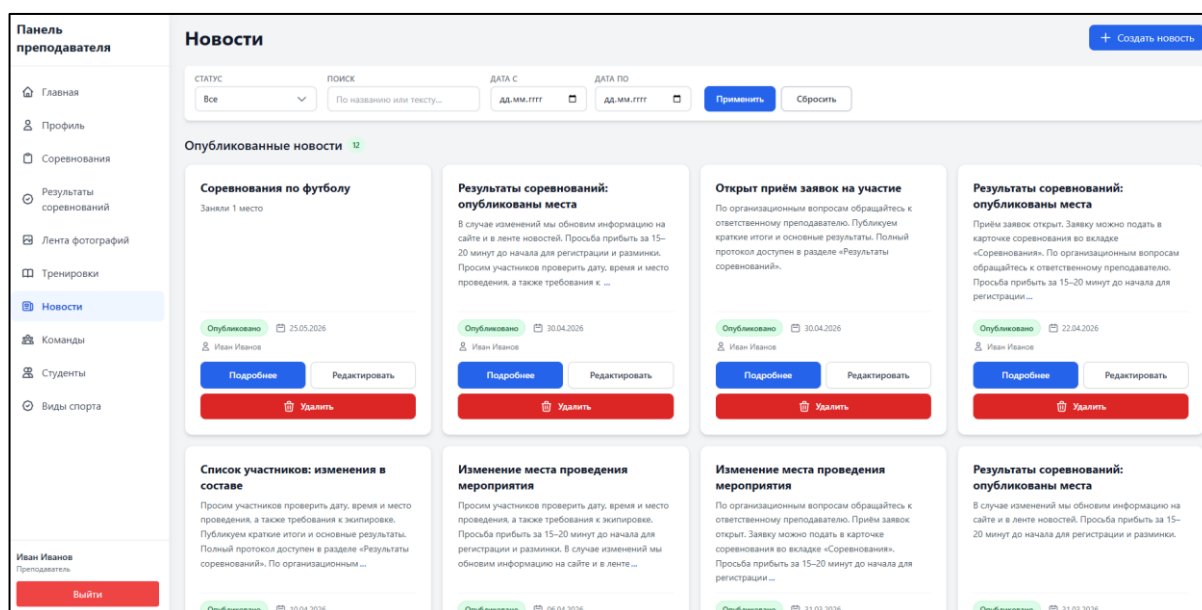


Рисунок 47 – Страница раздела новости

В разделе «Команды» отображается список команд и кнопка «Создать команду».

Карточка команды (преподаватель):

- Просмотр и правка состава.
- Добавление студентов (поиск по ФИО).
- Изменение роли участника в команде.
- Заявки на вступление – список ожидающих, кнопки одобрить / отклонить;
- Удаление участника из состава.

На рисунках 48-49 представлен раздел «Команды».

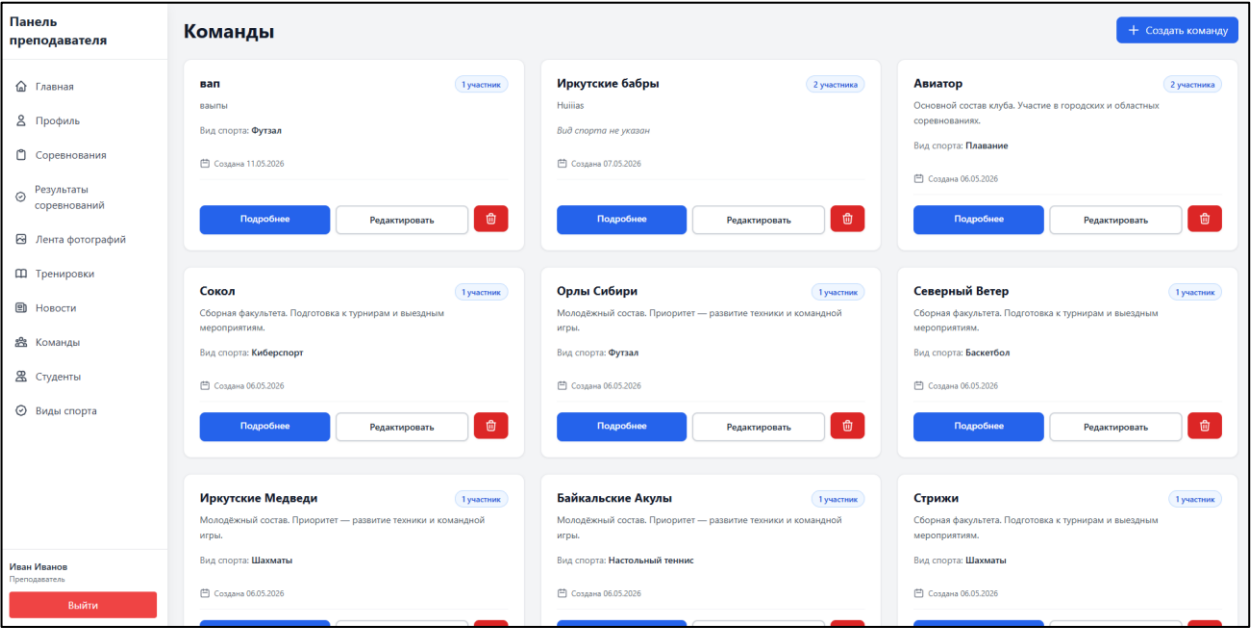


Рисунок 48 – Страница раздела «Команды»

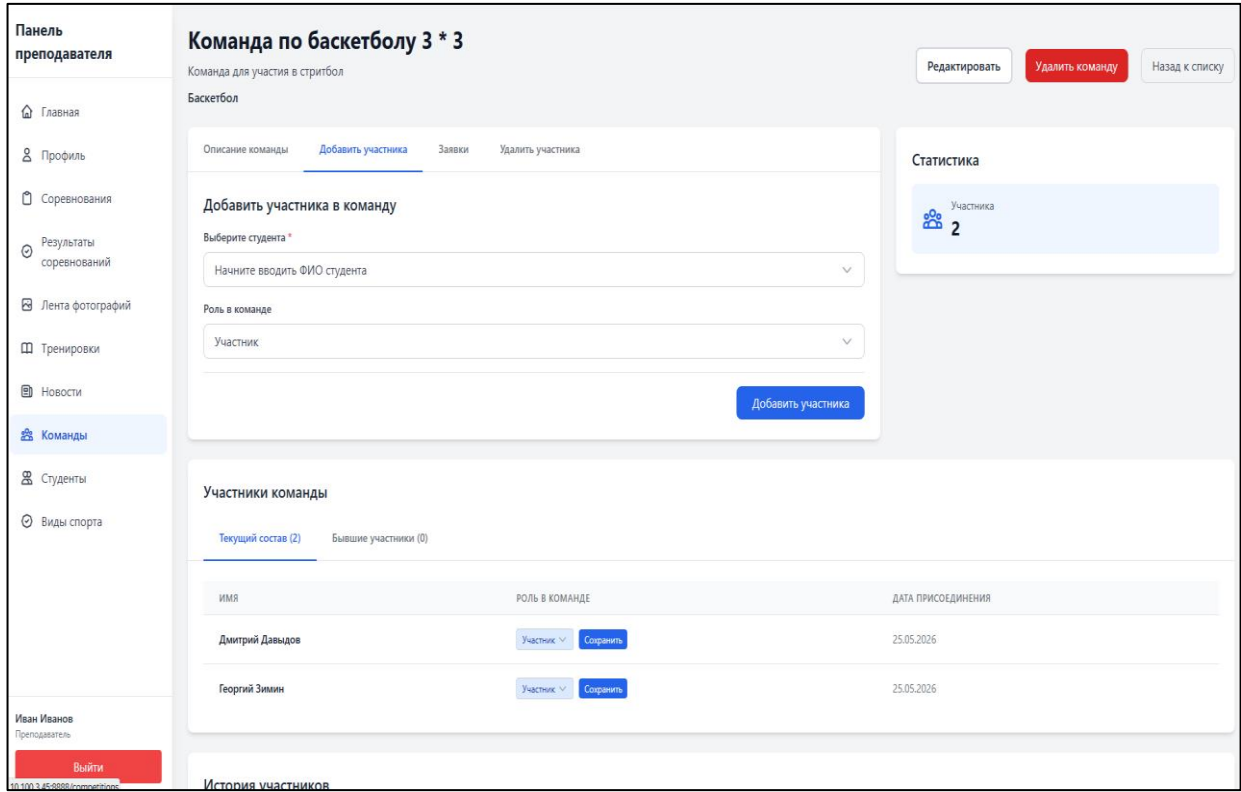


Рисунок 49 – Страница подробнее команды

По пункту «Студенты» открывается реестр студентов, сгруппированный по учебным группам.

Фильтры:

- поиск по фамилии;
- группа (все / конкретная / «Без группы»);
- статус физорга – все / физорги / не физорги.

В списке для каждого студента доступны переход в профиль и действие «Сделать физоргом» / «Убрать физорга» (в группе может быть один физорг).

В профиле студента – полные данные и кнопка назначения/снятия статуса физорга. Для группы доступен просмотр расписания.

На рисунках 50-51 представлен раздел «Студенты».

Панель преподавателя

Главная

Профиль

Соревнования

Результаты соревнований

Лента фотографий

Тренировки

Новости

Команды

Студенты

Виды спорта

Иван Иванов

Преподаватель

Выйти

Студенты

Фамилия

Начните вводить...

Группа: Все группы

Статус: ☒ Все ☐ Физорги ☐ Не физорги

Группа: ВЕБ-21-1 23

Расписание группы

ФАМИЛИЯ	ИМЯ	ОТЧЕСТВО	ЛОГИН	СТАТУС	ДЕЙСТВИЯ
Алексеев	Иван	Владиславович	21026	Студент	Профиль студента Сделать физоргом
Бондарь	Татьяна	Денисовна	21027	Студент	Профиль студента Сделать физоргом
Бородин	Лада	Александровна	21028	Студент	Профиль студента Сделать физоргом
Бякина	Татьяна	Андреевна	21029	Студент	Профиль студента Сделать физоргом
Власов	Егор	Алексеевич	21030	Студент	Профиль студента Сделать физоргом
Волокитин	Вячеслав	Сергеевич	21031	Студент	Профиль студента Сделать физоргом
Гачинская	Алиса	Вячеславовна	21032	Студент	Профиль студента Сделать физоргом
Захожева	Алина	Саидшарифовна	21033	Студент	Профиль студента Сделать физоргом
Казакова	Елизавета	Алексеевна	21034	Студент	Профиль студента Сделать физоргом
Мазенова	Полина	Вячеславовна	21035	Студент	Профиль студента Сделать физоргом
Медведев	Илья	Михайлович	21037	Студент	Профиль студента Сделать физоргом
Мельник	Алина	Дмитриевна	21038	Студент	Профиль студента Сделать физоргом
Мясников	Кирилл	Сергеевич	21039	Студент	Профиль студента Сделать физоргом

Рисунок 50 – Страница раздела «Студенты»

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была достигнута поставленная цель: разработано веб-приложение спортивного клуба «Авиатор» для ГБПОУИО «ИАТ», предназначенное для предоставления информации о спортивной деятельности техникума, автоматизации работы с тренировками, соревнованиями, командами и заявками пользователей.

В рамках дипломного проекта были решены следующие задачи:

- проведено предпроектное исследование предметной области, определены основные процессы, объекты, участники и их роли;
- выполнен анализ инструментальных средств, используемых при разработке программного продукта;
- разработано техническое задание на создание веб-приложения;
- выполнено проектирование программного продукта, включая архитектуру, функциональную структуру, базу данных и пользовательский интерфейс;
- реализованы база данных, серверная часть и клиентская часть веб-приложения;
- проведено функциональное и нагрузочное тестирование программного продукта;
- разработана пользовательская документация, включающая инструкцию по установке и руководство пользователя.

Разработанное веб-приложение обеспечивает просмотр новостей спортивного клуба, сведений о видах спорта, командах, тренировках, соревнованиях и результатах. Для студентов реализованы возможности записи на тренировки, подачи заявок на участие в соревнованиях и вступление в команды. Для преподавателей реализованы функции управления спортивной информацией, студентами, командами, тренировочными сессиями, соревнованиями, новостями и результатами.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 73
Изм.	Лист	№ докум.	Подпись	Дата		

Функциональное тестирование подтвердило корректность работы основных пользовательских сценариев, обработки некорректных данных и разграничения доступа по ролям. Нагрузочное тестирование позволило оценить стабильность работы веб-приложения при одновременных обращениях пользователей.

Практическая значимость проекта заключается в возможности использования разработанного веб-приложения для централизованного информирования студентов о спортивной жизни техникума, автоматизации организационных процессов спортивного клуба и упрощения учёта участия студентов в тренировках, командах и соревнованиях.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 74
Изм.	Лист	№ докум.	Подпись	Дата		

Список используемых источников

- 1 Blog.Skillfactory – Что такое PHP и для чего он нужен – URL: <https://blog.skillfactory.ru/glossary/php/>. (Дата обращения 16.02.2026). – Текст: электронный.
- 2 Getcomposer.org – Документация – URL: <https://getcomposer.org/doc/> (дата обращения: 10.01.2026) – Текст: электронный.
- 3 Habr.com – Юнит-тестирование в PHP / Хабр – URL: <https://habr.com/ru/articles/56289/> (Дата обращения 19.04.2026) – Текст: электронный.
- 4 iconify.design – библиотека бесплатных иконок – URL: <https://iconify.design/> (дата обращения 08.02.2026). – Текст электронный.
- 5 Laravel – Документация – URL: <https://laravel.su/docs/12.x/documentation> (дата обращения 12.01.2026). – Текст: электронный.
- 6 LdapRecord – Документация – URL: <https://ldaprecord.com/docs/laravel/v3> (дата обращения: 13.02.2026) – Текст: электронный.
- 7 MAX – Руководство для интеграции с мессенджером MAX – URL: <https://dev.max.ru/docs> (Дата обращения: 18.05.2026) – Текст: электронный.
- 8 Mockitt – Top 10 Admin UI Design Examples and How to Create It – URL: <https://mockitt.com/ui-ux-design/admin-ui-design.html> (дата обращения: 13.04.2026) – Текст: электронный.
- 9 MySQL – Документация – URL: <https://dev.mysql.com/doc/> (Дата обращения: 29.04.2026) – Текст: электронный.
- 10 npm – Документация – URL: <https://docs.npmjs.com/> (дата обращения: 16.01.2026) – Текст: электронный.
- 11 Parsing – Загрузка и парсинг CSV в Ларавел – URL: <https://laravel.demiart.ru/upload-and-parse-csv-in-laravel/> (Дата обращения: 28.04.2026) – Текст: электронный.

12 Swiper Demos – Демоверсии Swiper – URL: <https://swiperjs.com/demos> (Дата обращения: 12.02.2026) – Текст: электронный.

13 Tailwind CSS – Документация – URL: <https://tailwindcss.com/docs> (Дата обращения: 21.02.2026) – Текст: электронный.

14 WYSIWYG – Веб-конструктор WYSIWYG – URL: <https://www.wysiwygwebbuilder.com/> (Дата обращения: 14.04.2026) – Текст: электронный.

15 Хабр – Конвертируем HTML в PDF при помощи Dompfd – URL: <https://habr.com/ru/articles/190364/> (Дата обращения: 13.02.2026) – Текст: электронный.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист
						76
Изм.	Лист	№ докум.	Подпись	Дата		

Приложение А – Техническое задание

Министерство образования Иркутской области

Государственное бюджетное профессиональное образовательное учреждение
Иркутской области
«Иркутский авиационный техникум»
(ГБПОУИО «ИАТ»)

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

ВЕБ-ПРИЛОЖЕНИЯ

СПОРТИВНОГО КЛУБА «АВИАТОР»

Руководитель:

(П.Н. Чернигов)

(подпись, дата)

Студент:

(Г.М. Зимин)

(подпись, дата)

Иркутск 2026

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 77
Изм.	Лист	№ докум.	Подпись	Дата		

1 Введение

1.1 Общие сведения

Документ представляет собой техническое задание на создание веб-приложения спортивного клуба «Авиатор», предназначенный для автоматизации процессов спортивной жизни ГБПОУИО «ИАТ», включающий централизованное хранение и обработку сведений о пользователях, видах спорта, командах, тренировочных сессиях, соревнованиях, заявках на участие, новостях и результатах спортивных мероприятий.

Вид программного продукта: CRM/ERP.

1.2 Цели и задачи

Целью разработки веб-приложения спортивного клуба «Авиатор» является автоматизация процессов информирования пользователей о спортивной деятельности техникума, управления тренировками, соревнованиями, командами, заявками студентов и результатами спортивных мероприятий.

Программный продукт должен обеспечить централизованное хранение сведений о пользователях, видах спорта, командах, тренировочных сессиях, соревнованиях, заявках, новостях и результатах, а также предоставить пользователям инструменты для просмотра, поиска, фильтрации, подачи заявок и управления спортивной информацией в соответствии с назначенной ролью.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Реализовать модуль публичной информации, обеспечивающий просмотр новостей, видов спорта, команд и результатов соревнований неавторизованными пользователями.
2. Реализовать модуль авторизации пользователей и разграничения доступа на основе ролей «Студент» и «Преподаватель».

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 78
Изм.	Лист	№ докум.	Подпись	Дата		

3. Реализовать модуль профиля пользователя, обеспечивающий просмотр персональных данных и истории участия в спортивной деятельности.

4. Реализовать модуль управления тренировочными сессиями, обеспечивающий создание, редактирование, просмотр тренировок и запись студентов на тренировки.

5. Реализовать модуль управления соревнованиями, обеспечивающий создание, редактирование, просмотр соревнований и обработку заявок студентов на участие.

6. Реализовать модуль управления спортивными командами, обеспечивающий ведение состава команд и обработку заявок студентов на вступление в команды.

7. Реализовать модуль управления новостями, обеспечивающий создание, редактирование, публикацию и просмотр новостных материалов спортивного клуба.

8. Реализовать модуль справочников, обеспечивающий ведение сведений о видах спорта, категориях соревнований и локациях.

9. Реализовать модуль учёта результатов соревнований, обеспечивающий хранение и отображение итогов спортивных мероприятий, а также формирование отчета по соревнованиям за месяц.

10. Реализовать механизмы поиска, фильтрации, сортировки и постраничного вывода данных в основных разделах веб-приложения.

11. Реализовать тестирование основных функций веб-приложения и подготовить пользовательскую документацию.

2 Основания для разработки

2.1 Нормативные документы

Документ основывается на следующих нормативных документах:

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 79
Изм.	Лист	№ докум.	Подпись	Дата		

– ГОСТ 34.602-2020 "Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы".

– Методические указания по выполнению дипломного проекта по специальности 09.02.07 «Информационные системы и программирование», квалификация «специалист по информационным системам»;

– Методические указания по оформлению дипломного проекта по специальности 09.02.07 «Информационные системы и программирование»;

– Программа государственной итоговой аттестации по специальности 09.02.07 «Информационные системы и программирование», квалификация «специалист по информационным системам».

2.2 Проектные документы

Проектные документы включают:

- Задание на дипломное проектирование.
- Пояснительную записку.
- Руководство пользователя.

3 Назначение системы

3.1 Общее описание

Веб-приложение спортивного клуба «Авиатор» предназначено для автоматизации процессов организации спортивной деятельности в ГБПОУИО «ИАТ».

Система должна обеспечивать работу с новостями, видами спорта, командами, тренировочными сессиями, соревнованиями, заявками студентов и результатами спортивных мероприятий. Веб-приложение должно поддерживать просмотр спортивной информации, запись студентов на тренировки, подачу заявок на

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 80
Изм.	Лист	№ докум.	Подпись	Дата		

соревнования и вступление в команды, а также управление данными спортивного клуба пользователями с расширенными правами.

Доступ к функциям системы должен определяться ролью пользователя: «Неавторизованный пользователь», «Студент» или «Преподаватель».

4 Требования к системе

4.1 Функциональные требования

1 Роль «Неавторизованный пользователь»:

- Просмотр публичной новостной ленты спортивного клуба.
- Просмотр списка видов спорта в клубе.
- Возможность авторизации в веб-приложении.
- Просмотр команд и их публичных составов.
- Авторизация в веб-приложении.

2 Роль «Студент»:

- Просмотр новостной ленты.
- Просмотр портфолио.
- Просмотр расписания тренировок по видам спорта через веб-приложение.
- Получение уведомлений о новых тренировках в «Макс».
- Получение уведомлений о новых соревнованиях в «Макс».
- Запись на тренировку, с возможностью отмены записи.
- Просмотр ближайших соревнований.
- Подача заявки на соревнование с возможностью отмены.
- Просмотр состава команд спортивного клуба.
- Подача заявки на вступление в спортивную команду.
- Просмотр истории участия на соревнованиях, тренировках в личном кабинете.

3 Роль «Преподаватель»:

Управление студентами и командами:

- Добавление и удаление студентов в/из команды.
- Назначение роли студента в команде команды.
- Принятие заявки на вступление студента в команду.
- Управление тренировочным процессом:
- Создание, редактирование и удаление спортивных мероприятий в расписании(тренировок).

- Указание для тренировок: даты, времени, вида спорта, описания, названия, локации.

- Просмотр списка записавшихся на тренировку студентов.
- Отметка о прибытии студента на тренировку.

Управление новостями:

- Создание, редактирование и удаление новостных записей.
- Указание дополнительной информации для новости: даты, описание, фотографии.

Управление видами спорта:

- Добавление, редактирование и удаление видов спорта в спортивном клубе.

Управление спортивными соревнованиями:

- Добавление, удаление и редактирование спортивных соревнований.
- Указание дополнительной информации о соревновании: Дата проведения, вид соревнования, описание, локация, статус.
- Принятие заявок студентов на спортивные соревнования.
- Указание информации в соревновании о допуске к соревнованиям и выданной форме.
- Формирование приказов об освобождении студентов на соревнования.
- Просмотр архива тренировок, соревнований.

Управление результатами соревнований:

- Добавление фотографий с соревнования.
- Формирование отчета по соревнованиям за месяц.

4.2 Технические требования

4.2.1 Производительность

- Количество одновременных пользователей: до 50 активных сессий.
- Количество запросов в секунду (RPS/QPS): 20 RPS.
- Среднее время отклика: не более 500 мс.

4.2.2 Надежность

- Доступность системы не менее 99,8% в год.
- Валидация пользовательского ввода на клиенте – все формы, поля, файлы и параметры должны проверяться перед отправкой запроса на сервер.
- Валидация пользовательского ввода на сервере – все входные данные, независимо от наличия клиентской проверки, проходят повторную строгую валидацию.
- Защита от SQL–инъекций за счет применения ORM Eloquent с встроенной защитой и параметризованных запросов.

4.3 Эксплуатационные требования

1. Среда функционирования:
 - Поддержка современных браузеров: Яндекс браузер, Chrome, Microsoft Edge.
2. Установка и развёртывание:
 - Разработка инструкции по установке и запуску.
3. Интеграция:

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 83
Изм.	Лист	№ докум.	Подпись	Дата		

– Интеграция с системой каталогов LDAP (Active Directory) для аутентификации пользователей.

4.4 Требования к информационной безопасности

Веб-приложение должно обеспечивать защиту данных и предотвращение несанкционированного доступа в соответствии со следующими требованиями:

1. Аутентификация и авторизация:

– Система обеспечивает аутентификацию пользователей перед предоставлением доступа к функциональным модулям. Механизм аутентификации: через LDAP (Active Directory) с использованием логина и пароля.

– Пароли пользователей хранятся в зашифрованном виде в системе каталогов LDAP с использованием стойкого криптографического алгоритма, применяемого сервером каталогов.

– Для каждой пользовательской сессии формируется уникальный токен сессии. Токен имеет ограниченное время жизни (120 минут) и защищен от подделки. При выходе из системы токен уничтожается.

2. Защита от распространённых уязвимостей:

– Защита от несанкционированного доступа к данным – проверка прав доступа к каждому ресурсу на серверной стороне в зависимости от роли пользователя.

– Защита от межсайтовой подделки запросов (CSRF) – использование токенов CSRF для всех форм и операций изменения данных.

5 Требования к техническому обеспечению

5.1 Сервер

– Количество процессоров: 1 шт.

– Количество ядер одного процессора: 4 ядра.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 84
Изм.	Лист	№ докум.	Подпись	Дата		

- Тактовая частота одного процессора: 2.5 ГГц.
- Оперативная память: 16 Гб.
- Пропускная способность сети: 1 Гбит/с.

5.2 Хранилище данных

- Тип накопителей: SSD.
- Общий объем хранилища: 256 Гб.
- IOPS на чтение: 75000.
- IOPS на запись: 60000.

5.3 Клиентские устройства

5.3.1 Минимальные требования для персонального компьютера (ноутбука)

- Количество ядер процессора: 2 ядра.
- Тактовая частота процессора: 2.0 ГГц.
- Оперативная память: 8 Гб.
- Тип диска: HDD или SSD.
- Свободного места на диске: не менее 10 Гб.
- Разрешение дисплея: Full HD (1920×1080).
- Пропускная способность сети: 50 Мбит/с.

5.4 Сетевые требования

- Доступ к сети Интернет со скоростью не менее 100 Мбит/с.
- Поддержка следующих сетевых протоколов: HTTP/1.1, HTTP/2, WebSocket, DNS, TCP/IP, LDAP.
- Поддержка IPv4 (обязательно), IPv6 (при наличии).

6 Требования к программному обеспечению

6.1 Сервер

- Операционная система Ubuntu Server 24.04 LTS.
- Сервер базы данных: MySQL 8.4.7 LTS.
- Веб-сервер: Nginx 1.24.
- Интерпретатор: PHP 8.2+.
- Менеджер пакетов: Composer 2.7+ (для PHP).
- Система контроля версий: Git 2.45.

6.2 Персональный компьютер (ноутбук)

1. Операционная система Windows 10 / 11 (64-bit), Linux Ubuntu 22.04 / 24.04 LTS.
2. Веб-браузеры (актуальные версии, не старше двух релизов): Яндекс.Браузер 24.9+, Microsoft Edge 128+.
3. Необходимые компоненты браузера:
 - Поддержка HTML5, CSS3, ECMAScript 2015+ (ES6+).
 - Поддержка Web Storage API, IndexedDB, Fetch API.
 - Поддержка WebSocket, Service Workers, PWA.
 - Включена поддержка JavaScript и cookies.

7 Требования к тестированию

7.1 Общие требования

Тестирование веб-приложения должно охватывать все ключевые аспекты функциональности, производительности и безопасности. Этап тестирования (пункт 7.1, этап 4) должен включать:

- Документирование.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 86
Изм.	Лист	№ докум.	Подпись	Дата		

- Функциональное тестирование (минимум 3 функции).

7.1.1 Документирование

Тестовые сценарии (test cases) – охватывающие ключевые функциональные и нефункциональные требования к системе для проверки работы:

- в типовых и исключительных ситуациях;
- при вводе корректных и некорректных данных;
- при различных ролях пользователей (администратор, лаборант, старший лаборант);
- Тестовая документация должна включать:
 - план тестирования;
 - перечень тестовых сценариев;
 - отчёт о результатах тестирования и выявленных дефектах.

7.1.2 Функциональное тестирование

- Проверка корректности работы веб-приложения в соответствии с функциональными требованиями (пункт 4.1).
- Проверка обработки ошибок и сообщений пользователю при некорректных действиях.
- Проверка разграничения прав доступа в зависимости от роли пользователя.

8 Организационно-технические требования

8.1 Этапы разработки

В таблице 1 представлены сроки и этапы разработки веб-приложения.

					ДП.09.02.07-5.26.222.10.ПЗ	Лист 87
Изм.	Лист	№ докум.	Подпись	Дата		

Таблица 1 – Сроки и этапы разработки веб-приложения

№	Наименование этапов	Срок
1	Предпроектное исследование предметной области	до 22.02.26
2	Разработка технического задания	до 28.02.26
3	Проектирование программного обеспечения. (разработка структурной и функциональной схемы ПО, проектирование базы данных)	до 15.03.26
4	Разработка (программирование)	до 30.04.26
5	Тестирование и отладка	до 06.05.26
6	Составление программной документации	до 19.05.26

Приложение Б – Листинг web.php

```
<?php
use App\Http\Controllers\AuthController;
use App\Http\Controllers\CompetitionController;
use App\Http\Controllers\GuestController;
use App\Http\Controllers\HomeCarouselPhotoController;
use App\Http\Controllers\CompetitionCategoryController;
use App\Http\Controllers\LocationController;
use App\Http\Controllers\LocationTrainingController;
use App\Http\Controllers\NewsController;
use App\Http\Controllers\SportController;
use App\Http\Controllers\StudentController;
use App\Http\Controllers\StudentPortfolioController;
use App\Http\Controllers\TeamController;
use App\Http\Controllers\TeamJoinRequestController;
use App\Http\Controllers\TrainingSessionController;
use App\Http\Controllers\TrainingRegistrationController;
use App\Http\Middleware\EnsureAllowedRoles;
use App\Support\ParticipantListingDateFilter;
use App\Models\Competition;
use App\Models\Sport;
use App\Models\Team;
use App\Models\TrainingSession;
use Illuminate\Http\Request;
use Illuminate\Pagination\LengthAwarePaginator;
use Illuminate\Support\Facades\Route;
use Illuminate\Validation\Rule;
Route::get('/', function () {
    if (auth()->check()) {
        return redirect()->route('dashboard');}
    return app(GuestController::class)->index();
})->name('home');
Route::get('/guest/news', [GuestController::class, 'news']->name('guest.news');
Route::get('/guest/news/{news}', [GuestController::class, 'showNews']->
    >name('guest.news.show');
Route::get('/guest/sports', [GuestController::class, 'sports']->name('guest.sports');
Route::get('/guest/sports/{sport}', [GuestController::class, 'showSport']->
    >name('guest.sports.show');
Route::get('/guest/teams', [GuestController::class, 'teams']->name('guest.teams');
Route::get('/guest/teams/{team}', [GuestController::class, 'showTeam']->
    >name('guest.teams.show');
```

					ДП.09.02.07-5.26.222.10.ПЗ	Лист
						89
Изм.	Лист	№ докум.	Подпись	Дата		

```

Route::get('/guest/competitions', [GuestController::class, 'competitions'])->name('guest.competitions');
Route::get('/guest/competitions/{competition}', [GuestController::class, 'showCompetition'])->name('guest.competitions.show');
Route::get('/guest/results', [GuestController::class, 'results'])->name('guest.results');
Route::get('/guest/training-sessions', [GuestController::class, 'trainingSessions'])->name('guest.training-sessions');
Route::get('/guest/training-sessions/{trainingSession}', [GuestController::class, 'showTrainingSession'])->name('guest.training-sessions.show');
Route::get('/guest/users/{user}', [GuestController::class, 'showUserProfile'])->name('guest.users.show');
Route::middleware('guest')->group(function () {
Route::get('/login', [AuthController::class, 'showLoginForm'])->name('login');
Route::post('/login', [AuthController::class, 'login']);
});
Route::match(['get', 'post'], '/logout', [AuthController::class, 'logout'])->name('logout');
Route::middleware(['auth', EnsureAllowedRoles::class])->group(function () {
Route::get('/dashboard', function () {
$user = auth()->user();
$publishedNews = \App\Models\News::with(['creator', 'images'])
->where('status', 'Published')
->latest('date')
->take(3)
->get();
$upcomingCompetitions = \App\Models\Competition::with(['sport', 'team', 'location', 'images'])
->whereIn('status', ['upcoming', 'ongoing'])
->where('end_date', '>=', now()->toDateString())
->orderBy('start_date', 'asc')
->take(3)
->get();
$upcomingTrainingSessions = \App\Models\TrainingSession::with(['sport', 'team', 'location'])
->where('status', '!=', 'cancelled')
->where('status', '!=', 'completed')
->where('end_time', '>=', now())
->orderBy('start_time', 'asc')
->take(3)
->get();
$view = $user->role === 'teacher' ? 'dashboard-teacher' : 'dashboard-student';
return view($view, [
'user' => $user,
'publishedNews' => $publishedNews,

```

```

'upcomingCompetitions' => $upcomingCompetitions,
'upcomingTrainingSessions' => $upcomingTrainingSessions]);
})->name('dashboard');
Route::get('/profile', function () {
    $user = auth()->user();
    $view = $user->role === 'teacher' ? 'profile-teacher' : 'profile-student';
    $portfolioCount = null;
    if ($user && $user->role === 'student') {
        $portfolioCount = (int) \App\Models\CompetitionResult::query()
        ->whereIn('place', ['1', '2', '3'])
        ->where(function ($q) use ($user) {
            $q->where(function ($p) use ($user) {
                $p->where('result_type', 'personal')
                ->where('user_id', $user->id);
            })->orWhere(function ($t) use ($user) {
                $t->where('result_type', 'team')
                ->whereHas('competition.participants', function ($pp) use ($user) {
                    $pp->where('user_id', $user->id);});});});
        ->whereHas('competition', function ($c) {
            $c->where('status', 'finished');});
        ->count();}
    return view($view, ['user' => $user, 'portfolioCount' => $portfolioCount]);
})->name('profile');
Route::get('/profile/avatar/edit', [AuthController::class, 'editAvatar'])->
    >name('profile.avatar.edit');
Route::post('/profile/avatar', [AuthController::class, 'updateAvatar'])->
    >name('profile.avatar');
Route::middleware([EnsureAllowedRoles::class . ':student'])->group(function () {
    Route::get('/profile/portfolio', [StudentPortfolioController::class, 'index'])->
        >name('profile.portfolio');
    Route::get('/profile/portfolio/competitions/{competition}',
        [StudentPortfolioController::class, 'show'])
        ->name('profile.portfolio.show');
    Route::get('/profile/participations/teams', function () {
        $user = auth()->user();
        $memberships = $user->teamMembersForProfileParticipationListing()
        ->sortByDesc(fn ($m) => $m->joined_at?->timestamp ?? 0)
        ->values()
        return view('profile.participations.teams', compact('user', 'memberships'));
    })->name('profile.participations.teams');
    Route::get('/profile/participations/competitions', function (Request $request) {
        $listingFilters = $request->validate([
            'sport_id' => ['nullable', 'integer', Rule::exists('sports', 'id')],

```

```

'date_from' => ['nullable', 'date_format:Y-m-d'],
'date_to' => ['nullable', 'date_format:Y-m-d'],]);
$user = auth()->user();
$sportId = isset($listingFilters['sport_id']) ? (int) $listingFilters['sport_id'] : null;
$dateFrom = $listingFilters['date_from'] ?? null;
$dateTo = $listingFilters['date_to'] ?? null;
$filteredCompetitions = $user->competitionParticipants()
->with(['competition.sport', 'competition.team', 'competition.location'])
->get()
->filter(function ($p) use ($sportId, $dateFrom, $dateTo) {
    $c = $p->competition;
    if (!$c) {
        return false;
    }
    if ($c->status !== 'finished') {
        return false;
    }
    if ($sportId && (int) $c->sport_id !== $sportId) {
        return false;
    }
    if (! ParticipantListingDateFilter::competitionIntervalsOverlap(
        $c->start_date,
        $c->end_date,
        $dateFrom,
        $dateTo
    )) {return false;}return true;})
->sortByDesc(function ($p) {
    $c = $p->competition;
    return optional($c->end_date)->timestamp
    ?? optional($c->start_date)->timestamp
    ?? 0;})
->values();
$perPage = 20;
$page = LengthAwarePaginator::resolveCurrentPage();
$participations = new LengthAwarePaginator(
    $filteredCompetitions->slice(($page - 1) * $perPage, $perPage)->values(),
    $filteredCompetitions->count(),
    $perPage,
    $page,
    ['path' => $request->url(), 'query' => $request->query()]);
$sportsForFilter = Sport::orderBy('name')->get();
return view('profile.participations.competitions', compact(
    'user',
    'participations',
    'listingFilters',
    'sportsForFilter'

```

```

));})->name('profile.participations.competitions');
Route::get('/profile/participations/trainings', function (Request $request) {
    $listingFilters = $request->validate([
        'sport_id' => ['nullable', 'integer', Rule::exists('sports', 'id')],
        'date_from' => ['nullable', 'date_format:Y-m-d'],
        'date_to' => ['nullable', 'date_format:Y-m-d'],]);
    $user = auth()->user();
    $sportId = isset($listingFilters['sport_id']) ? (int) $listingFilters['sport_id'] : null;
    $dateFrom = $listingFilters['date_from'] ?? null;
    $dateTo = $listingFilters['date_to'] ?? null;
    $filteredRegistrations = $user->trainingRegistrations()
        ->with(['training.sport', 'training.team', 'training.location'])
        ->latest('registered_at')
        ->get()
        ->filter(function ($r) use ($sportId, $dateFrom, $dateTo) {
            $t = $r->training;
            if (! $t || ! $t->isParticipantHistoryFinished()) {
                return false;
            }
            if ($sportId && (int) $t->sport_id !== $sportId) {
                return false;
            }
            if (! ParticipantListingDateFilter::trainingSessionIntervalsOverlap(
                $t->start_time,
                $t->end_time,
                $dateFrom,
                $dateTo
            )) {return false;}return true;})
        ->sortByDesc(fn ($r) => $r->training?->end_time?->timestamp ?? 0)
        ->values();
    $perPage = (int) $request->query('per_page', 20);
    if (! in_array($perPage, [10, 25, 50, 100], true)) {
        $perPage = 20;
    }
    $page = LengthAwarePaginator::resolveCurrentPage();
    $registrations = new LengthAwarePaginator(
        $filteredRegistrations->slice(($page - 1) * $perPage, $perPage)->values(),
        $filteredRegistrations->count(),
        $perPage,
        $page,
        ['path' => $request->url(), 'query' => $request->query()]);
    $sportsForFilter = Sport::orderBy('name')->get();
    return view('profile.participations.trainings', compact('user', 'registrations',
        'listingFilters', 'sportsForFilter'));
    })->name('profile.participations.trainings'));});
Route::get('/teams', function (Request $request) {

```

```

$user = auth()->user();
if ($user->role === 'teacher') {
return app(TeamController::class)->index($request);} else {
return app(TeamController::class)->indexStudent($request);
}}->name('teams.index')->middleware(['auth', EnsureAllowedRoles::class]);
Route::get('/competitions', function (Request $request) {
$user = auth()->user();
if ($user->role === 'teacher') {
return $controller->index($request);
} else {return $controller->indexStudent($request);
}}->name('competitions.index')->middleware(['auth', EnsureAllowedRoles::class]);
Route::get('/competitions/my', [CompetitionController::class, 'myCompetitions'])
->name('competitions.student.my')
->middleware(['auth', EnsureAllowedRoles::class . ':student']);
Route::get('/competitions/results', [CompetitionController::class, 'results'])-
>name('competitions.results')->middleware(['auth', EnsureAllowedRoles::class]);
Route::get('/training-sessions', function (Request $request) {
$controller = new TrainingSessionController();
$user = auth()->user();
if ($user->role === 'teacher') {
return $controller->index($request);
} else {return $controller->indexStudent($request);}
})->name('training-sessions.index')->middleware(['auth', EnsureAllowedRoles::class]);
Route::get('/training-sessions/student', function (Request $request) {
return app(TrainingSessionController::class)->indexStudent($request);
})->name('training-sessions.student.index')->middleware(['auth',
EnsureAllowedRoles::class . ':student']);
Route::get('/training-sessions/my', [TrainingSessionController::class, 'myTrainings'])
->name('training-sessions.student.my')
->middleware(['auth', EnsureAllowedRoles::class . ':student']);
Route::middleware(['auth', EnsureAllowedRoles::class . ':teacher'])->group(function () {
Route::get('/competitions/create', [CompetitionController::class, 'create'])-
>name('competitions.create');
Route::get('/competitions/results/report', [CompetitionController::class,
'resultsReport'])->name('competitions.results.report');
Route::get('/competitions/photo-archive', [CompetitionController::class,
'photoArchive'])->name('competitions.photo-archive');
Route::delete('/competitions/photo-archive/{competitionImage}',
[CompetitionController::class, 'destroyCompetitionPhoto'])->name('competitions.photo-
archive.destroy');
Route::get('/training-sessions/create', [TrainingSessionController::class, 'create'])-
>name('training-sessions.create');

```

```

Route::get('/my/news', [NewsController::class, 'myIndex'])->name('news.my');
Route::get('/my/competitions', [CompetitionController::class, 'myIndex'])->name('competitions.my');
Route::get('/my/sports', [SportController::class, 'myIndex'])->name('sports.my');});
Route::get('/competitions/{competition}', function (Competition $competition) {
    $controller = new CompetitionController();
    $user = auth()->user();
    if ($user->role === 'teacher') {
        return $controller->show($competition);
    } else {return $controller->showStudent($competition);}
})->name('competitions.show')->middleware(['auth', EnsureAllowedRoles::class]);
Route::get('/training-sessions/{trainingSession}', function (TrainingSession $trainingSession) {
    $controller = new TrainingSessionController();
    $user = auth()->user();
    if ($user->role === 'teacher') {
        return $controller->show($trainingSession);} else {
        return $controller->showStudent($trainingSession);
    }}->name('training-sessions.show')->middleware(['auth', EnsureAllowedRoles::class]);
Route::middleware(['auth', EnsureAllowedRoles::class . ':student'])->group(function () {
    Route::post('/training-sessions/{trainingSession}/register',
    [TrainingRegistrationController::class, 'register'])->name('training-sessions.register');
    Route::post('/training-sessions/{trainingSession}/unregister',
    [TrainingRegistrationController::class, 'unregister'])->name('training-sessions.unregister');
    Route::post('/competitions/{competition}/apply', [CompetitionController::class, 'applyStudent'])->name('competitions.apply');
    Route::post('/guest/teams/{team}/join-requests', [TeamJoinRequestController::class, 'store'])->name('guest.teams.join-requests.store');
    Route::post('/teams/{team}/join-requests', [TeamJoinRequestController::class, 'store'])->name('teams.join-requests.store');});
Route::get('/news', function () {
    $controller = new NewsController();
    $user = auth()->user();
    if ($user->role === 'teacher') {
        return $controller->index(request());
    } else {return $controller->indexStudent(request());}
})->name('news.index')->middleware(['auth', EnsureAllowedRoles::class]);
Route::middleware(['auth', EnsureAllowedRoles::class . ':teacher'])->group(function () {
    Route::get('/teams/create', [TeamController::class, 'create'])->name('teams.create');
    Route::post('/teams', [TeamController::class, 'store'])->name('teams.store');
    Route::get('/teams/{team}/edit', [TeamController::class, 'edit'])->name('teams.edit');
    Route::put('/teams/{team}', [TeamController::class, 'update'])->name('teams.update');

```

```

Route::patch('/teams/{team}', [TeamController::class, 'update']);
Route::delete('/teams/{team}', [TeamController::class, 'destroy'])->
>name('teams.destroy');
Route::post('/teams/{team}/members', [TeamController::class, 'addMember'])->
>name('teams.members.add');
Route::get('/teams/{team}/search-students', [TeamController::class, 'searchStudents'])->
>name('teams.search-students');
Route::put('/teams/{team}/members/{member}', [TeamController::class,
'updateMemberRole'])->name('teams.members.update-role');
Route::post('/teams/{team}/members/{member}/accept', [TeamController::class,
'acceptMember'])->name('teams.members.accept');
Route::delete('/teams/{team}/members/{member}', [TeamController::class,
'removeMember'])->name('teams.members.remove');
Route::get('/students', [StudentController::class, 'index'])->name('students.index');
Route::get('/students/groups/{groupName}/schedule', [StudentController::class,
'groupSchedule'])->name('students.groups.schedule');
Route::post('/students/{student}/toggle-fizorg', [StudentController::class,
'toggleFizorg'])->name('students.toggle-fizorg')->whereNumber('student');
Route::get('/students/{student}', [StudentController::class, 'show'])->
>name('students.show')->whereNumber('student');
Route::get('/teams/{team}/join-requests', [TeamJoinRequestController::class, 'index'])->
>name('teams.join-requests.index');
Route::post('/teams/{team}/join-requests/{joinRequest}/approve',
[TeamJoinRequestController::class, 'approve'])->name('teams.join-requests.approve');
Route::post('/teams/{team}/join-requests/{joinRequest}/reject',
[TeamJoinRequestController::class, 'reject'])->name('teams.join-requests.reject');
Route::get('/home-carousel-photos', [HomeCarouselPhotoController::class, 'index'])->
>name('home-carousel-photos.index');
Route::post('/home-carousel-photos', [HomeCarouselPhotoController::class, 'store'])->
>name('home-carousel-photos.store');
Route::put('/home-carousel-photos/order', [HomeCarouselPhotoController::class,
'updateOrder'])->name('home-carousel-photos.order');
Route::delete('/home-carousel-photos', [HomeCarouselPhotoController::class,
'destroy'])->name('home-carousel-photos.destroy');
Route::resource('sports', SportController::class);
Route::resource('news', NewsController::class)->except(['index', 'show']);
Route::post('/news/{news}/publish', [NewsController::class, 'publish'])->
>name('news.publish');
Route::resource('competitions', CompetitionController::class)->except(['index', 'show',
'create']);
Route::post('/competitions/{competition}/cancel', [CompetitionController::class,
'cancel'])->name('competitions.cancel');

```



```

Route::post('/competitions/{competition}/results', [CompetitionController::class,
'storeResult'])->name('competitions.results.store');
Route::put('/competitions/{competition}/results/{result}', [CompetitionController::class,
'updateResult'])->name('competitions.results.update');
Route::delete('/competitions/{competition}/results/{result}',
[CompetitionController::class, 'destroyResult'])->name('competitions.results.destroy');
Route::get('/competitions/{competition}/photos', [CompetitionController::class,
'photos'])->name('competitions.photos');
Route::post('/competitions/{competition}/photos', [CompetitionController::class,
'storePhotos'])->name('competitions.photos.store');
Route::resource('training-sessions', TrainingSessionController::class)->except(['index',
'show', 'create']);
Route::post('/training-sessions/{trainingSession}/cancel',
[TrainingSessionController::class, 'cancel'])->name('training-sessions.cancel');
Route::post('/location-trainings', [LocationTrainingController::class, 'store'])->
>name('location-trainings.store');
Route::put('/location-trainings/{locationTraining}', [LocationTrainingController::class,
'update'])->name('location-trainings.update');
Route::delete('/location-trainings/{locationTraining}',
[LocationTrainingController::class, 'destroy'])->name('location-trainings.destroy');
Route::get('/locations', [LocationController::class, 'index'])->name('locations.index');
Route::post('/locations', [LocationController::class, 'store'])->name('locations.store');
Route::put('/locations/{location}', [LocationController::class, 'update'])->
>name('locations.update');
Route::delete('/locations/{location}', [LocationController::class, 'destroy'])->
>name('locations.destroy');
Route::get('/competition-categories', [CompetitionCategoryController::class, 'index'])->
>name('competition-categories.index');
Route::post('/competition-categories', [CompetitionCategoryController::class, 'store'])->
>name('competition-categories.store');
Route::put('/competition-categories/{category}', [CompetitionCategoryController::class,
'update'])->name('competition-categories.update');
Route::delete('/competition-categories/{category}',
[CompetitionCategoryController::class, 'destroy'])->name('competition-
categories.destroy');
Route::post('/competitions/{competition}/applications/{application}/accept',
[CompetitionController::class, 'acceptCompetitionApplication'])->
>name('competitions.applications.accept');
Route::post('/competitions/{competition}/applications/{application}/reject',
[CompetitionController::class, 'rejectCompetitionApplication'])->
>name('competitions.applications.reject');
Route::post('/competitions/{competition}/participants', [CompetitionController::class,
'addParticipant'])->name('competitions.participants.add');

```

```

Route::put('/competitions/{competition}/participants/{userId}/role',
[CompetitionController::class, 'updateParticipantRole'])-
>name('competitions.participants.update-role');
Route::delete('/competitions/{competition}/participants/{user}',
[CompetitionController::class, 'removeParticipant'])-
>name('competitions.participants.remove');
Route::get('/competitions/{competition}/search-students',
[CompetitionController::class, 'searchStudents'])->name('competitions.search-students');
Route::get('/competitions/{competition}/search-teachers',
[CompetitionController::class, 'searchTeachers'])->name('competitions.search-
teachers');
Route::post('/competitions/{competition}/teacher', [CompetitionController::class,
'updateResponsibleTeacher'])->name('competitions.teacher.update');
Route::post('/competitions/{competition}/forms', [CompetitionController::class,
'storeForms'])->name('competitions.forms.store');
Route::post('/competitions/{competition}/form-types', [CompetitionController::class,
'storeFormType'])->name('competitions.form-types.store');
Route::post('/competitions/{competition}/medical-admission',
[CompetitionController::class, 'storeMedicalAdmission'])-
>name('competitions.medical-admission.store');
Route::post('/competitions/{competition}/generate-order-1',
[CompetitionController::class, 'generateOrder1'])->name('competitions.generate-order-
1');
Route::post('/competitions/{competition}/generate-order-2',
[CompetitionController::class, 'generateOrder2'])->name('competitions.generate-order-
2');
Route::post('/competitions/{competition}/generate-order-3',
[CompetitionController::class, 'generateOrder3'])->name('competitions.generate-order-
3');
Route::post('/competitions/{competition}/generate-named-application',
[CompetitionController::class, 'generateNamedApplication'])-
>name('competitions.generate-named-application');});
Route::get('/news/{news}', [NewsController::class, 'show'])->name('news.show')-
>middleware(['auth', EnsureAllowedRoles::class]);
Route::get('/teams/{team}', function (Team $team) {
$user = auth()->user();
if ($user->role === 'teacher') {
return app(TeamController::class)->show($team);} else {
return app(TeamController::class)->showStudent($team);
}})->name('teams.show')->middleware(['auth', EnsureAllowedRoles::class]);});

```